Graph Topology Augmentation for Prioritized Sweeping in Non-stationary Reinforcement Learning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Prioritized Sweeping (PS) accelerates model-based reinforcement learning by selecting backups according to Bellman residual magnitude. In nonstationary reward settings, however, the canonical priority score is shortsighted: after a localized reward shift, residuals propagate only through realized backups, so bottlenecked or topologically distant state estimates may remain static under a limited replanning budget. We introduce **Graph Topology Augmentation for Prioritized Sweeping** (GTA-PS), a drop-in ordering score, located for tabular model-based planners with fixed dynamics and changing state rewards. GTA-PS constructs a soft policy-induced transition graph and augments the key with a mixing of regularized directed-Laplacian potentials that diffuses residual information through forward and backward graph structure. The topology weight is annealed by a scheduler based on the Second Largest Eigenvalue Modulus (SLEM), allowing the queue to adapt to the chain's mixing regime. We prove that the forward potential coincides with geometric discounted residual propagation and show that GTA-PS gives nonzero priority instantly to states that standard PS can leave blocked after sparse reward shifts. Tabular experiments on FourRooms and GARNET domains demonstrate improved replanning efficiency over standard PS under both exact DP and Dyna-style host planners.

1 Introduction

Model-based reinforcement learning (MBRL) improves general behavior by using a known or learned transition model for planning [58, Ch. 8]. The agent benefits most when planning effort can be directed to the states that matter most, especially within a brittle planning budget. [57, 29]. **Prioritized Sweeping** (PS, or *Standard-PS* as to distinguish from our introduced variants) addresses this by maintaining a queue of states keyed by Bellman or Temporal-Difference residual magnitude [40, 43]. For each planning step, the state with the largest priority has its value estimate updated. After that, predecessors' value estimates often increase in magnitude; their refreshed priorities have higher chance to be queried next. This effect encourages information propagation backward through the predecessor support graph. Although conceptually simple, the resulting sweep can improve update efficiency by several orders of magnitude compared to uniform random backups [64].

We focus our study in the case of nonstationary Markov Decision Process (MDP), where the transition dynamics are fixed but the state reward changes over time [42]. This setting appears whenever the navigation structure is comparatively stable but the task objective changes, as in goal-conditioned robotics [44], dynamic pricing and operations settings [5], and sequential recommendation problems [66]. The standard approach using residual priority is *shortsighted* and *reactive*, as most state order remain stale until the Bellman wave has propagated to them through concrete realized backups. Under the budget of K updates between reward shifts, some topologically distant states may never be reached, regardless of how sophisticated the host planner is [58]. In the nonstationary setting

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

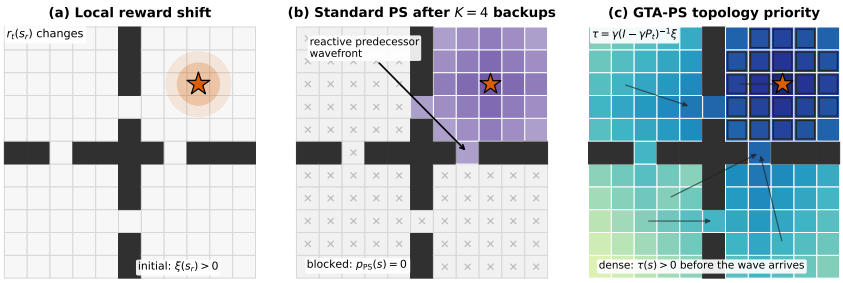


Figure 1: A localized reward shift creates sparse residual at s_r. Standard PS is shortsighted: priority of distant states receives no information hints. GTA-PS adds a topology potential on the policy-induced graph, assigning priority to upstream states before predecessor pushes reach them.

of our focus, this creates a further hindsight effect: by the time the residual signal reaches distant regions, the reward information that triggered it may already be stale. The resulting value errors can then feed back into the planning–action loop, causing the host to allocate future updates according to outdated priorities, compound into a circular degradation of both planning quality and subsequent data collection [14, 31].

We propose to address the former hindrance – credit propagation – by utilizing the model as assignment guidance. The **Graph Topology Augmentation** (GTA) framework capture preferences of the agent's navigation field using spectral operators. From that, we derive rich representations and signal preconditioning that can be used to augment available score-based decisions, such as credit assignment, or bandit identification. **Graph Topology Augmentation for Prioritized Sweeping** (GTA-PS) is an implementation of the GTA ideology for our nonstationary setting study, that augment the residual-keyed priority with an additive mixed terms constructed from directed Laplacian inverses. The method is broadly applicable to orthogonal techniques, such as functional approximation, partial observability, and model uncertainty.

Our contributions.

- We formalize priority-based planning as a host-agnostic queue-management problem, making the priority definition a modular design choice. The same GTA-PS module plugs into both exact tabular (DP) and Dyna-style planners without modifying the host.
- We introduce **Graph Topology Augmentation** (GTA), a general RL framework that applies spectral operators on policy-induced chains to produce topology-aware planning signals.
- We present **GTA-PS**, a concrete GTA instantiation for nonstationary rewards. It augments residual priority with a directed regularized-Laplacian potential and a fixed forward/backward weight aggregation together with a SLEM-driven β_t scheduler that adapts topology gain to the chain's mixing regime.
- We prove key properties of the topology signal, including its numerical stability, the geometric discount and implicit importance sampling effects, plus SLEM adaptive capabilities.
- We present a succint experimental evaluation, variating chain statistics across environments and planner architectures, with surgical ablations isolating each cause's effectiveness.

2 Related Work

In model-based planning, three lines of work have been established to provide richer planning guidance beyond raw residual magnitude, aiding prioritized sweeping decision. *Successor-based signals* spread value information over the state graph rather than waiting for purely local error cascades [21, 56]. *Learned abstractions* compress the planning space, utilizing the generalization capabilities of supervised learning, as in *VaST*'s abstract tabular model [19]. *Graph-structural heuristics* uses auxiliary graph representation, as in *TVI/FTVI*, which exploit the *strongly connected components* decomposition of the MDP [20]. Together these demonstrate that good planning requires more than raw residuals and that model knowledge is a powerful source of leadership.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Spectral graph theory has a long history of applications in RL representation learning. *Proto-value functions* [36, 37] use undirected Laplacian eigenvectors as smooth value-approximation bases [53, 6]. The same eigenvectors underpin *option discovery* [34] and its deep analogues via the *successor representation* [35] and *cover-time options* [30]. Scalable computational methods include *Krylov-subspace methods* such as Arnoldi and GMRES [26, 46], *Graph-drawing objectives* with unique solutions [62], and recent *exact recovery methods* [27]. Recent concurrent work on *effective resistance* has proposed it as an intrinsic reward guiding agents toward structurally accessible goals [13]. In these works, spectral operators mainly serve as offline representation or filtering bases rather than as a direct mechanism for shaping online backup selection.

Our proposed **Graph Topology Augmentation** framework brings analytical Markov-chain tools into an RL planning module. It builds on the Laplacian-potential intuition from electrical networks [23], treating the policy-induced chain as the Laplacian kernel [16] and its Green's function as the potential operator [17]. Eigenvalues of the directed chain [32] is utilized as a scheduling signal. Additionally, several analytical tools such as biorthogonal decomposition [28], additive reversibility decomposition [11], and classical spectral graph bounds [18] are investigated to further enrich the framework.

3 Preliminary

Markov Decision Processes. We consider a finite discounted MDP

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r_t, \gamma),$$

with $|\mathcal{S}| < \infty$, $|\mathcal{A}| < \infty$, fixed discount $\gamma \in (0, 1)$, and a *fixed* transition kernel $p(s' \mid s, a)$. The reward is a bounded, state-based vector $r_t : \mathcal{S} \to \mathbb{R}$ that **may change over time**. Non-station is introduced in the modeling process as demand permutations captured by the reward signal, while the navigation skeleton is asserted to be static.

Priority-based planning. A planning sweep can only be realistically implemented on a serial computer as a sequence of individual backups. Such updates poses to be more effective when performed in-place, so that later backups in the same sweep can immediately use earlier updates [58, Ch.4]. Hence, one can go even further, by applying an ordering on backups, so that the overlapping effect is most utilized. Such order can be maintained programmatically in a priority queue, ensuring the queried element always hold the largest weight. The particular landscape of priority-based planning is more sophisticated, and we dedicate to describe them thoroughly in [B.3, D.5].

Reward setting. Let $\{\tau_m\}_{m \geq 0}$ denote the reward-shift times and let $N_{\text{shift}}(n)$ be the number of shifts up to step n. The reward is piecewise stationary on each interval $[\tau_m, \tau_{m+1})$ and may shift exogenously at the times τ_m with maximum magnitude Δ_r.

exogenously = ngoại sinh (sự dịch chuyển reward là do môi trường)

Sparse or localized reward changes are the main regime of interest. Let $e_s \in \mathbb{R}^{|\mathcal{S}|}$ denote the standard basis vector associated with state s, i.e.,

$$(e_s)_{s'} = \begin{cases} 1, & s' = s, \\ 0, & s' \neq s. \end{cases}$$

A rank-one reward shift at time t is a localized perturbation of the form $\delta \mathbf{r}_t = \Delta_{r,t}\, e_{s_{r,t}}$ where $s_{r,t} \in \mathcal{S}$ is the affected state and $\Delta_{r,t}$ is the shift amplitude. A shift batch aggregates multiple localized reward shifts occurring at the same time step:

$$\delta \mathbf{r}_t = \sum_{i=1}^{m_t} \Delta_{r,t}^{(i)}\, e_{s_{r,t}^{(i)}}, \quad m_t \leq m_{\max} \ll |\mathcal{S}|.$$

s_{r,t} là trạng thái bị thay đổi phần thưởng tại thời điểm t.
delta_{r,t} là biên độ thay đổi phần thưởng tại thời điểm t.

This holds in almost all logistical reward design settings.

Agent setting. The host planner can vary: exact asynchronous value iteration or a Dyna-style planning loop. In both cases the planner maintains a value estimate, requiring efficient backup ordering. Such individual backup is denoted as a *planning step*, planned before the host decides to take a committal action (denoted as a *host step*). In our paper, the number of planning step is fixed to K per host step for fair comparisons. Our setting deliberately limits the use of prior knowledge, so that the agent is designed under minimal assumptions. The information available at initialization is either an oracle tabular model in the DP setting, or an immature learned model in the learning setting. Details of component settings in each host algorithm is discussed in B.1.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

D_t^in = diag(P_t^T.1)

Table 1: Core notation used.

Symbol	Meaning
$\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r_t, \gamma)$	Finite discounted tabular MDP with stable transition skeleton and piecewise-stationary reward
π, π^{PS}	Host policy and topology policy, respectively
P_t^{PS} (e.g., P_t)	Topology-policy-induced transition matrix
$L_t^{\mathrm{out}} = I - P_t$	Out-Laplacian; governs forward diffusion
$L_t^{\mathrm{in}} = D_t^{\mathrm{in}} - (P_t)^\top$	In-Laplacian; governs backward attribution
$\mathbf{f}_t^{\mathrm{out}}$, $\mathbf{f}_t^{\mathrm{in}}$	Forward and backward residual potentials
$\mathrm{SLEM}(P_t)$ (e.g., SLEM)	Second largest eigenvalue modulus of the topology chain
K, T_{shift}	Fixed planning budget per host step and host step budget per reward shift respectively

T_shift = $\tau_{m+1} - \tau_m

Residuals as inferred belief. Residual (in short for *value estimation residual*) measures the discrepancy between the planner's current estimate and new, implied one by latest evidence. Formally,

$$\xi_t(s) := |\mathcal{T}_t V_t(s) - V_t(s)|,$$

where $\mathcal{T}_t V_t(s)$ is the value estimate inferred from the current reward and transition information. Prioritized sweeping treats residual magnitude as a belief-correction score: states with larger discrepancies are backed up first. The important caution is that the inferred estimation should be both *unbiased* and *robust* to variance, which limits the residual utilizing only one-step samples.

Paper-wise scope. In our theoretical interest, we lean on the ideal ground of an oracle model and tabular value function as the main analytical regime. For general usage (specifically in the Dyna deployment), the efficiency of our proposal is verified empirically [Sec. 6].

4 Graph Topology Augmentation for Prioritized Sweeping

4.1 Graph Topology Augmentation framework

Policy-induced random walk. For any policy $\mu(a \mid s)$, the induced transition matrix

$$P^\mu = \sum_a \mathrm{diag}(\mu(\cdot, a))\, P^a, \qquad P^\mu(s, s') = \sum_a \mu(a \mid s)\, p(s' \mid s, a),$$

defines a random walk operator on the state graph. GTA treats P^μ as the soft adjacency matrix of the MDP that captures both the transitional rule and the navigational preference of the policy.

Directed topology operators. The row-stochastic matrix P_t defines a directed state graph. GTA-PS uses two regularized directed Laplacian systems:

$$L_t^{\mathrm{out}} = I - P_t, \qquad L_t^{\mathrm{in}} = D_t^{\mathrm{in}} - P_t^\top, \qquad D_t^{\mathrm{in}} = \mathrm{diag}(P_t^\top \mathbf{1}).$$

The out-potential measures how much residual a walk starting from s expects to encounter under the topology chain. The in-potential applies the analogous construction on the edge-reversed flow. For a residual vector ξ_t, define the diffused signal

$$\mathbf{f}_t^{\mathrm{out}} = (L_t^{\mathrm{out}} + \eta I)^{-1} |\xi_t|, \qquad \mathbf{f}_t^{\mathrm{in}} = (L_t^{\mathrm{in}} + \eta I)^{-1} |\xi_t|, \qquad \eta > 0.$$

where the regularizer ηI makes both systems invertible A.2.1.

The forward potential has a particularly grounded interpretation. At the canonical regularization $\eta^\star = \frac{1-\gamma}{\gamma}$. Since $L_t^{\mathrm{out}} = I - P_t$, $(L_t^{\mathrm{out}} + \eta^\star I)^{-1} = \gamma(I - \gamma P_t)^{-1}$. Therefore

$$\mathbf{f}_t^{\mathrm{out}} = \gamma(I - \gamma P_t)^{-1} \xi_t = \gamma \sum_{k \geq 0} (\gamma P_t)^k \xi_t,$$

$$\Rightarrow f_t^{\mathrm{out}}(s) = \gamma\, \mathbb{E}_{P_t}\left[\sum_{k \geq 0} \gamma^k \xi_t(X_k) \mid X_0 = s \right].$$

Thus the diagonal regularizer is not an arbitrary feature: η effectively controls the geometric discount horizon. With $\eta = \eta^\star$, it equals exactly the reward discount rate γ. Some additional diffusers design also hold various forms of effective horizon, as discussed further in C.1.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Second Largest Eigenvalue Modulus. For a general stochastic matrix P, the eigenvalues are ordered by modulus as $1 = \lambda_1 \geq |\lambda_2| \geq \cdots \geq |\lambda_n|$. As the operator is applied iteratively, the Second Largest Eigenvalue Modulus $\mathrm{SLEM}(P) = |\lambda_2|$ governs the asymptotic contraction rate of the chain toward its stationary distribution [32]. When $\mathrm{SLEM}(P_t)$ is close to 1, the chain has slow modes that can cause long delays in reward propagation – often occurs in bottlenecks [55]. This reveals the potential of annealing learning process with SLEM, pays as a minor compute overhead.

Graph Topology Augmentation framework As canonical planning techniques uses mixing techniques or representational kernels to artificially transmit the *reactive* signal, our GTA framework instead takes a *topological-driven* approach – informing decision processes by preconditioning signal with topological operator. The framework facilitates four roles:

- proactive priority signal for guiding the algorithm 4.2;
- detector of high-impact states and bottlenecks E.2;
- scheduler for modulating the topology gain 4.2;
- diagnostic tool for understanding the planning geometry A.2.

4.2 The GTA-PS instantiation.

Topology policy. As to guide the priority credit process, we design the *topology policy* π_t^{PS}, separated but dependent on the host's policy. For the convergence behavior of standard VI to hold [Lemma A.2.4], the choice of π_t^{PS} is not critically constrained, as long as it is reasonably aligned with the host's policy and ensures infinite-visit exploration [Lemma 5.1.1]. We propose a soft policy based on the softmax function. Define

$$Q_t(s,a) = r_t(s) + \gamma \sum_{s' \in \mathcal{S}} P_t^a(s,s') V_t(s')$$

as the one-step action-value estimate on (s,a). Define the centered action score

$$A_t(s,a) = Q_t(s,a) - \max_{b \in \mathcal{A}} Q_t(s,b).$$

We feed this as logits to construct the Boltzmann policy over action advantages with global unit-normalizer temperature constant $b > 0$:

$$\pi_t^{\mathrm{PS}}(a \mid s) = \frac{\exp(b\, A_t(s,a))}{\sum_{a'} \exp(b\, A_t(s,a'))}, \qquad P_t = \sum_a \mathrm{diag}(\pi_t^{\mathrm{PS}}(\cdot,a))\, P^a.$$

This keeps the ordering of action preferences but softens the policy, ensuring infinite explorations requirement in 5.1 Finite b standardizes the logits scale-free.

Augmented priority score. Standard PS uses the Bellman residual $p_t^{\mathrm{PS}}(s) = |\xi_t(s)|$ as its queue key. GTA-PS keeps this residual key and adds a structure-aware correction. The module produces a topology-imbued signal:

$$\mathbf{f}_t^{\mathrm{out}} = (L_t^{\mathrm{out}} + \eta I)^{-1} |\xi_t|, \qquad \mathbf{f}_t^{\mathrm{in}} = (L_t^{\mathrm{in}} + \eta I)^{-1} |\xi_t|,$$

$\mathbf{f}_t^{\mathrm{out}}(s)$ quantifies the forward topological influence of the residual at s, and $\mathbf{f}_t^{\mathrm{in}}(s)$ quantifies the backward influence from predecessors via the edge-reversed graph. These potentials are the standard graph-theoretic solutions to $(L_t^{\mathrm{out}} + \eta I)\mathbf{f}_t^{\mathrm{out}} = \xi_t$ and $(L_t^{\mathrm{in}} + \eta I)\mathbf{f}_t^{\mathrm{in}} = \xi_t$.

The topology signal is:

$$\boldsymbol{\tau}_t \triangleq (1-\alpha_t) \left|\mathbf{f}^{\mathrm{in}}\right| + \alpha_t \left|\mathbf{f}^{\mathrm{out}}\right|$$

where $\alpha_t \equiv \alpha \in (0,1)$ is held fixed in deployed variants, balancing forward and backward information.

GTA-PS replaces the queue key with:

$$\mathrm{priority}_t(s) = |\xi_t(s)| + \beta_t\, \tau_t(s).$$

$\xi_t(s)$ is the reactive base, while $\tau_t(s)$ is the proactive topology correction.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 1 Pseudocode for GTA-PS computational workloads

Input: value estimate V, reward r, model $\{\bar{P}^a\}_{a\in\mathcal{A}}$, predecessor map Pred, backup budget K, refresh period M, frontier size q
Parameters: topology temperature b, mix $\alpha \in [0,1]$, regularizer $\eta > 0$, scheduler parameters $\beta_1 > 0, \beta_2 \geq 0$
Convention: $\bar{P}^a = P^a$ in exact DP and $\bar{P}^a = \widehat{P}^a$ in Dyna.

A. Priority refresh

procedure REFRESHKEYS($V, r, \{\bar{P}^a\}$)
  Build $\pi^{\mathrm{PS}}(a|s) \propto \exp(bA(s,a))$
  $P \leftarrow \sum_a \mathrm{diag}(\pi^{\mathrm{PS}}(\cdot,a))\,\bar{P}^a$
  $L^{\mathrm{out}} \leftarrow I - P$
  $D^{\mathrm{in}} \leftarrow \mathrm{diag}(P^\top \mathbf{1})$
  $L^{\mathrm{in}} \leftarrow D^{\mathrm{in}} - P^\top$
  $\xi(s) \leftarrow |(\mathcal{T}_{r,\bar{P}}V)(s) - V(s)|, \quad \forall s \in \mathcal{S}$
  Solve $(L^{\mathrm{out}} + \eta I) f^{\mathrm{out}} = \xi$
  Solve $(L^{\mathrm{in}} + \eta I) f^{\mathrm{in}} = \xi$
  $\tau \leftarrow (1-\alpha)|f^{\mathrm{out}}| + \alpha|f^{\mathrm{in}}|$
  $\sigma \leftarrow \mathrm{SLEM}(P)$
  $\beta \leftarrow \beta_1(1 - \sigma + \beta_2)$
  $\mathcal{F}_q \leftarrow \mathrm{TOPQ}(\tau, q)$
  $\mathcal{Q}_\xi \leftarrow \mathrm{HEAP}_{s:\xi(s)>0}[\xi(s) + \beta\tau(s)]$
  $\mathcal{H}_q \leftarrow \mathrm{HEAP}_{s\in\mathcal{F}_q}[\xi(s) + \beta\tau(s)]$
  return $(P, \xi, \tau, \beta, \mathcal{F}_q, \mathcal{Q}_\xi, \mathcal{H}_q)$
end procedure

B. Budgeted prioritized planning

procedure GTA-PS($V, r, \{\bar{P}^a\}, K$)
  $(P, \xi, \tau, \beta, \mathcal{F}_q, \mathcal{Q}_\xi, \mathcal{H}_q) \leftarrow$ REFRESHKEYS($V, r, \{\bar{P}^a\}$)
  for $k = 1, \ldots, K$ **do**
    $s \leftarrow$ POPMAX($\mathcal{Q}_\xi, \mathcal{H}_q$)
    if $s = \varnothing$ **then**
      break
    end if
    Remove s from both heaps
    $v_{\mathrm{old}} \leftarrow V(s)$
    $V(s) \leftarrow (\mathcal{T}_{r,\bar{P}}V)(s)$
    $\Delta V \leftarrow V(s) - v_{\mathrm{old}}$
    for $u \in \{s\} \cup \mathrm{Pred}(s)$ **do**
      $\xi(u) \leftarrow |(\mathcal{T}_{r,\bar{P}}V)(u) - V(u)|$
      UPDATE($\mathcal{Q}_\xi, u, \xi(u) + \beta\tau(u)$)
      if $u \in \mathcal{F}_q$ **then**
        UPDATE($\mathcal{H}_q, u, \xi(u) + \beta\tau(u)$)
      end if
    end for
    if REWARDSHIFTED() or $k \bmod M = 0$ **then**
      $(P, \xi, \tau, \beta, \mathcal{F}_q, \mathcal{Q}_\xi, \mathcal{H}_q) \leftarrow$ REFRESHKEYS($V, r, \{\bar{P}^a\}$)
    end if
  end for
  return V
end procedure

Two-queue architecture with top-q frontier. Because $\boldsymbol{\tau}_t = G\xi_t$ is dense (nonzero everywhere when $\xi_t \neq \mathbf{0}$), inserting it directly into the residual heap would crowd the queue with low-priority states, adding an excessive $O(|\mathcal{S}|)$ cost multiplier. The simple solution is to limit the queue capacity by q. Additionally, the implementation uses two heaps: $\mathcal{Q}_\xi$ over the nonzero residual set and $\mathcal{H}_q$ over a filtered top-q frontier extracted from the dense $\mathcal{Q}_\tau$. This *decouples the two signals*: the residual heap guarantees that states with large Bellman error remain eligible at all times, while the topology frontier is retained only as a fallback once the residual queue is empty. Setting $q_t = |\mathcal{S}|$ removes the frontier truncation but does not alter the residual gate.

SLEM-based β scheduler Should we anneal α_t β_t to react fairly to the MDP flow and reward non-station? As α_t should intuitively be hold constant for a fair credit assignment (and empirically so C.2), we introduce variations in β_t.

In our called GTA-PS$^\beta$ variant, we use a SLEM-driven linear scheduler,

$$\beta_t = \beta_1(1 - \mathrm{SLEM}(P_t) + \beta_2),$$

which amplifies topology gain when $1 - \mathrm{SLEM}(P_t) + \beta_2 > 0$ and dampens it otherwise. This scheduler adaptively reduce topology gain in slowly mixing regimes, preventing long-horizon propagation from overexploiting stale or noisy residual signals in early stages.

The intuition for these design choices is further expanded in Appendix C.

5 Theoretical Analysis of GTA-PS

General identities and convergence statements below use the standing assumptions from Section 4. Results about blocked states and cumulative tracking additionally require bounded, sparse, ergodic reward shifts, as made explicit in the theorem statement and Appendix A.1. All full proofs are given in Appendix A.3.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

5.1 Convergence-guarantee of GTA-PS on DP

As to ensure GTA-PS is not worse than standard PS, Value Iteration should retain converge to the optimal policy under stationary MDP condition – value estimation can always recover if the shifts are sparse enough. We state central properties that govern this, providing the decay factor $\gamma < 1$.

Lemma 5.1.1 (Convergence under infinite state visits)**.** *Let $\mathcal{T}$ be the Bellman optimality operator, a γ-contraction in $\|\cdot\|_\infty$. Suppose an asynchronous backup scheme selects a sequence of states $\{s_k\}_{k\geq 0}$ such that every state $s \in \mathcal{S}$ is selected infinitely often. Then $V_k \to V^*$ in $\|\cdot\|_\infty$, independently of the backup-ordering rule.*

We claim that GTA-PS maintains lifelong exploration:

Proposition 5.1 (GTA-PS satisfies the infinite-visit condition)**.** *Under GTA-PS$^\beta$ with $\beta_2 > 0$, $\beta_1(1+\beta_2) < \eta^*$, and random tie-breaking among equal-priority states in the queue, then every state $s \in \mathcal{S}$ is selected for backup infinitely often. Consequently, GTA and GTA-PS$^\beta$ converges to V^* in $\|\cdot\|_\infty$.*

5.2 Operator Theory and Spectral Geometry

We provide analytical tools for controlling the resolvent interaction.

Lemma 5.2.1 (Green's function spectral representation)**.** *Given that the kernel P_t is diagonalizable (Cor. 1) with a biorthogonal eigendecomposition $P_t = \sum_k \lambda_k v_k u_k^\top$ and $u_j^\top v_k = \delta_{jk}$. Then for any $\eta > 0$, the forward potential has the spectral expansion*

$$\mathbf{f}_t^{out} = \sum_k \frac{u_k^\top |\xi_t|}{1 - \lambda_k + \eta}\, v_k,$$

so slow modes are amplified by at most $1/\eta$, and the first non-Perron mode $k = 2$ is the SLEM-associated topology-discriminating component.

Proposition 5.2 (Doob h-transform and implicit importance sampling)**.** *Fix a single-site shift at s_r and define the Green's predecessor entry $\psi(s) \triangleq G^{out}_{\eta^\star,t}(s, s_r) > 0$. Then*

$$\tilde{P}_t(s, s') \triangleq \frac{\gamma P_t(s, s')\, \psi(s')}{\psi(s)}$$

*is the Doob h-transform of the γ-killed topology chain. Along any path $(X_0, \ldots, X_k)$, the associated change-of-measure weight telescopes as $\psi(X_k)/\psi(X_0)$. In this sense, ranking states by $\psi(s)$ is an implicit **importance-sampling** rule over propagation paths toward the shifted reward site.*

Proposition 5.3 (Dense support and PS barrier)**.** *Assume a single-site reward shift at s_r and let*

$$\mathcal{B}_K \triangleq \{s \in \mathcal{S} : h^*(s; s_r) > K\}$$

be the blocked set beyond the K-backup predecessor wavefront. Then: (i) $\tau_t(s) > 0$ for every state s; and (ii) after at most K reactive PS backups, every blocked state satisfies $p_t^{\mathrm{PS}}(s) = 0$ while $\tau_t(s) > 0$. This reveals the reactive barrier, alongside bottleneck hindrance effect formally.

5.3 Nonstationary Performance Analysis

Finally, we present cumulative, inter-episode superiority of GTA-PS$^\beta$ over standard PS. We apply the assumption 4 that the evolving topology kernel remains close to a frozen reference kernel $\bar{P}$:

$$\|P_t - \bar{P}\|_\infty \leq \varepsilon_{\mathrm{pol}}.$$

Lemma 5.3.1 (Lipschitz priority mass advantage)**.** *For a fixed single-site episode-start residual field $\zeta = \Delta_r e_{s_r}$, define the proactive reward shift vector:*

$$\tau(P, \zeta) \triangleq \gamma(I - \gamma P)^{-1}\zeta.$$

Let $\tau_{(q+1)}(P, \zeta)$ denote the $(q+1)$-st largest entry of $\tau(P, \zeta)$, using the conservative convention that ties at the cutoff are excluded from the top-q set. Define

$$\mathrm{adv}(P, \zeta) \triangleq \frac{1}{\gamma} \sum_{s \in \mathcal{B}_K(s_r)} \big(\tau(P, \zeta)(s) - \tau_{(q+1)}(P, \zeta)\big)_+$$

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

as the excess priority mass, within the K-neighborhood of the reward-shift site, above the top-q cutoff.

Then, $\mathrm{adv}(P,\zeta)$ *is uniformly Lipschitz in* P*:*

$$|\mathrm{adv}(P,\zeta) - \mathrm{adv}(P',\zeta)| \le L\,\|P - P'\|_\infty, \qquad L \triangleq \frac{2|\mathcal{S}|\gamma\Delta_{\max}}{(1-\gamma)^2}.$$

Proposition 5.4 (Cumulative ergodic advantage)**.** *Under additional assumptions 3 and 5*

$$\frac{1}{N}\sum_{t=1}^{N} \mathrm{adv}_t(P_t,\zeta_t) \ge \bar{A} - L\varepsilon_{\mathrm{pol}} + o(1) \qquad \textit{a.s.},$$

for some $\bar{A} > 0$*.*

Theorem 5.1 (Cumulative tracking advantage of GTA-PS over reactive PS)**.** *Under additional assumptions 2, with budget* $q = K$*. Assume also that* $|\mathcal{B}_K^{\mathrm{sel},t}| \ge 1$ *at positive* μ_R*-frequency. Define* $\mathrm{err}_t^{\mathrm{GTA}} \triangleq \tau_{(q+1),t}/\gamma$*,* $\mathrm{TE}^{\mathrm{GTA}}(N) \triangleq \sum_{t=1}^{N} \mathrm{err}_t^{\mathrm{GTA}}$*, and*

$$\mathrm{TE}^{\mathrm{PS}}(N) \triangleq \sum_{t=1}^{N} \sum_{s^* \in \mathcal{B}_K^{\mathrm{sel},t}} \Phi_t(s^*),$$

where $\Phi_t \triangleq (I - \gamma P_t)^{-1}\xi_t$*. Then the cumulative tracking error of GTA-PS is strictly smaller than reactive PS by at least* $N(\bar{A} - L\varepsilon_{\mathrm{pol}}) + o(N)$ *almost surely. In particular, if* $L\varepsilon_{\mathrm{pol}} < \bar{A}$ *then* $\mathrm{TE}^{\mathrm{PS}}(N) - \mathrm{TE}^{\mathrm{GTA}}(N) \to +\infty$ *almost surely.*

Significance. Thm. 5.1 is the centerpiece of our analysis: it provides the first cumulative guarantee for topology-augmented planning, showing that GTA-PS's advantage over reactive PS grows without bound as the number of episodes increases.

6 Experiments

We evaluate GTA-PS on tabular GARNET [4] and Minigrid FourRooms [15] under repeated nonstationary reward shifts, applying an oracle DP host and a Dyna host with a fixed planning budget K per host step. Full environment setup, reward protocol, hyperparameters, and computational details are described to Appendix D.1–D.7.

6.1 Recovery and planning efficiency

The main empirical result is that GTA-PS improves both planning accuracy, reward collection and shift recovery under the same planning budget. In the Dyna setting of Fig. 2, both GTA variants reduce value error faster than Standard-PS across all tested state sizes and budgets, with the gain most visible immediately after each shift when reactive propagation is still local. Cumulative performance also holds. Fig. 3 shows that the per-episode error gap accumulates into a persistent advantage (Fig. 8), matching the interpretation of Thm. 5.1. These results support the practical claim of greater sample efficiency under limited replanning.

6.2 Robustness and Recommendations

Ablations on fixed, varied $\alpha = \alpha_t$ and $\beta = \beta_t$ in Appendix D.8 show that moderate β improves the agreement between the topology field and the realized pop order, while oversized β sharply degrades. This is consistent with the convergence logic of Prop. 5.1: the topology term is most useful as a bias that surfaces blocked states early and not as a replacement for residual-driven prioritization. Complementary α sweeps also favor small backward mixing. In practice we therefore tune GTA-PS with small α and moderate β.

Extended stochasticity, transport, and bottleneck audits are investigated to Appendix D.8. The common lesson is environment adaptability: GTA-PS remains effective in stochastic GARNET instances and in bottlenecked FourRooms layouts, but a custom, highly adversarial transport can render topological guidance clueless. The best of both worlds is to keep topology guidance as an adaptive bias rather than a hard replacement for residual priority.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

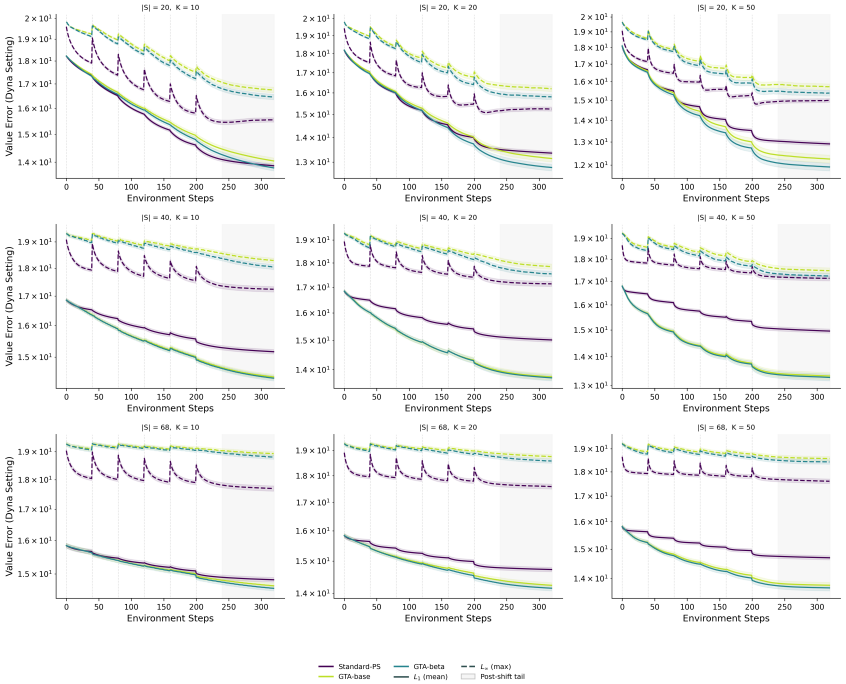


Figure 2: Repeated reward-shift recovery under a Dyna host. Each panel reports value-error traces after multiple reward shifts for a fixed planning budget K and state size $|\mathcal{S}|$. Solid lines denote mean L_1 error and dashed lines denote mean L_∞ error. Across all tested sizes and budgets, both GTA variants recover faster than Standard-PS, with GTA-PS$^\beta$ typically matching or slightly improving over the fixed-β variant.

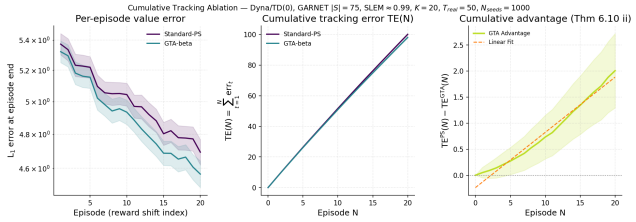


Figure 3: Cumulative tracking ablation under repeated reward shifts in GARNET. GTA-PS$^\beta$ lowers the per-episode terminal error, reduces cumulative tracking error $\mathrm{TE}(N)$, and produces a steadily increasing cumulative advantage over Standard-PS. This is the theorem-facing empirical evidence for persistent advantage under frequent reward shifts.

7 Conclusion

Our GTA framework utilizes graph-theoretic foundations to craft representation operators that capture entities of interest. By leveraging these tools, GTA therefore can capture subtle patterns of connectivity, influence, and community structure that are not visible from simple degree counts or path-based metrics. On top of this spectral foundation, GTA-PS employs learned models to map beliefs in both the model and the policy into a rich structural priority score. Within the technique, we provide intensive theoretical analysis and surgical experiments to reveals its profound ground. We hope to further extend the framework to other settings, as well as combine it with orthogonal advances in representation learning, exploration, and scalable planning.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

References

[1] David Abel. simple_rl: Reinforcement learning in Python, 2019. URL https://github.com/david-abel/simple_rl.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701. URL https://doi.org/10.1145/3292500.3330701.

[3] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper_files/paper/2017/file/453fadbd8a1a3af50a9df4df899537b5-Paper.pdf.

[4] T. W. Archibald, K. I. M. McKinnon, and L. C. Thomas. On the generation of Markov decision processes. *Journal of the Operational Research Society*, 46(3):354–361, 1995. doi: 10.1057/jors.1995.55.

[5] Yossi Aviv and Amit Pazgal. A partially observed Markov decision process for dynamic pricing. *Management Science*, 51(9):1400–1416, 2005. doi: 10.1287/mnsc.1050.0393.

[6] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003. doi: 10.1162/089976603321780317.

[7] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[8] Abraham Berman and Robert J. Plemmons. *Nonnegative Matrices in the Mathematical Sciences*, volume 9 of *Classics in Applied Mathematics*. SIAM, reprint edition, 1994. doi: 10.1137/1.9781611971262.

[9] Dimitri P. Bertsekas and John N. Tsitsiklis. *Parallel and Distributed Computation: Numerical Methods*. Prentice Hall, Englewood Cliffs, NJ, 1989.

[10] Rajendra Bhatia. *Matrix Analysis*, volume 169 of *Graduate Texts in Mathematics*. Springer, 1997. doi: 10.1007/978-1-4612-0653-8.

[11] Joris Bierkens. Non-reversible metropolis-hastings. *Statistics and Computing*, 26(6):1213–1228, 2016.

[12] Blai Bonet and Héctor Geffner. Labeled RTDP: Improving the convergence of real-time dynamic programming. In *Proceedings of the 13th International Conference on Automated Planning and Scheduling (ICAPS)*, pages 12–21. AAAI Press, 2003.

[13] Jatin Chauhan, Shivam Bhardwaj, Aditya Saibewar, Aditya Ramesh, Sadbhavana Babar, and Manohar Kaul. Graph-theoretic intrinsic reward: Guiding RL with effective resistance. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://openreview.net/forum?id=W8bKDPf1Ko.

[14] Wang Chi Cheung, David Simchi-Levi, and Ruihao Zhu. Reinforcement learning for non-stationary Markov decision processes: The blessing of (more) optimism. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pages 1843–1854. PMLR, 2020. URL https://arxiv.org/abs/1906.02922.

[15] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Doina Precup. Minigrid & Miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2306.13831.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

[16] Fan Chung. Laplacians and the Cheeger inequality for directed graphs. *Annals of Combinatorics*, 9(1):1–19, 2005. doi: 10.1007/s00026-005-0237-z.

[17] Fan Chung and Shing-Tung Yau. Discrete Green's functions. *Journal of Combinatorial Theory, Series A*, 91(1-2):191–214, 2000. doi: 10.1006/jcta.2000.3094.

[18] Fan R. K. Chung. *Spectral Graph Theory*. Number 92 in CBMS Regional Conference Series in Mathematics. American Mathematical Society, 1997.

[19] Dane Corneil, Wulfram Gerstner, and Johanni Brea. Efficient model-based deep reinforcement learning with variational state tabulation. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1049–1058. PMLR, 2018. URL https://arxiv.org/abs/1802.04325.

[20] Peng Dai, Mausam, Daniel S. Weld, and Judy Goldsmith. Topological value iteration algorithms. *Journal of Artificial Intelligence Research*, 42:181–209, 2011. doi: 10.1613/jair.3390. URL https://arxiv.org/abs/1401.3910.

[21] Peter Dayan. Improving generalization for temporal difference learning: The successor representation. *Neural Computation*, 5(4):613–624, 1993. doi: 10.1162/neco.1993.5.4.613.

[22] Thomas G. Dietterich. Hierarchical reinforcement learning with the MAXQ value function decomposition. *Journal of Artificial Intelligence Research*, 13:227–303, 2000. doi: 10.1613/jair.773.

[23] Peter G. Doyle and J. Laurie Snell. *Random Walks and Electric Networks*, volume 22 of *Carus Mathematical Monographs*. Mathematical Association of America, 1984. URL https://arxiv.org/abs/math/0001057.

[24] Stanley C. Eisenstat, Howard C. Elman, and Martin H. Schultz. Variational iterative methods for nonsymmetric systems of linear equations. *SIAM Journal on Numerical Analysis*, 20(2):345–357, 1983. doi: 10.1137/0720023.

[25] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23 (98):298–305, 1973.

[26] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

[27] Diego Gomez, Michael Bowling, and Marlos C. Machado. Proper Laplacian representation learning. *CoRR*, 2023. URL https://arxiv.org/abs/2310.10833.

[28] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 1985.

[29] Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. URL https://arxiv.org/abs/1906.08253.

[30] Yuu Jinnai, Jee Won Park, David Abel, and George Konidaris. Discovering options for exploration by minimizing cover time. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 3130–3139. PMLR, 2019.

[31] Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75:477–524, 2022. doi: 10.1613/jair.1.13673.

[32] David A. Levin and Yuval Peres. *Markov Chains and Mixing Times*. American Mathematical Society, 2nd edition, 2017.

[33] David A. Levin, Yuval Peres, and Elizabeth L. Wilmer. *Markov Chains and Mixing Times*. American Mathematical Society, 2009.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

[34] Marlos C. Machado, Marc G. Bellemare, and Michael Bowling. A Laplacian framework for option discovery in reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 2295–2304. PMLR, 2017. URL https://proceedings.mlr.press/v70/machado17a.html.

[35] Marlos C. Machado, Clemens Rosenbaum, Xiaoxiao Guo, Miao Liu, Gerald Tesauro, and Murray Campbell. Eigenoption discovery through the deep successor representation. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1710.11089.

[36] Sridhar Mahadevan. Proto-value functions: Developmental reinforcement learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 553–560, 2005. doi: 10.1145/1102351.1102421.

[37] Sridhar Mahadevan and Mauro Maggioni. Proto-value functions: A Laplacian framework for learning representation and control in Markov decision processes. *Journal of Machine Learning Research*, 8:2169–2231, 2007.

[38] J. A. Meijerink and H. A. van der Vorst. An iterative solution method for linear systems of which the coefficient matrix is a symmetric M-matrix. *Mathematics of Computation*, 31(137): 148–162, 1977. doi: 10.1090/S0025-5718-1977-0438681-4.

[39] Ravi Montenegro and Prasad Tetali. Mathematical aspects of mixing times in Markov chains. *Foundations and Trends in Theoretical Computer Science*, 1(3):237–354, 2006. doi: 10.1561/0400000003.

[40] Andrew W. Moore and Christopher G. Atkeson. Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning*, 13(1):103–130, 1993. doi: 10.1007/BF00993104.

[41] Nuitka Team. Nuitka: The python compiler, 2024. URL https://nuitka.net/.

[42] Sindhu Padakandla, Prabuchandran K. J, and Shalabh Bhatnagar. Reinforcement learning algorithm for non-stationary environments. *Applied Intelligence*, 50:3590–3606, 2020. doi: 10.1007/s10489-020-01758-5.

[43] Jing Peng and Ronald J. Williams. Efficient learning and planning within the Dyna framework. *Adaptive Behavior*, 1(4):437–454, 1993. doi: 10.1177/105971239300100403.

[44] Matthias Plappert, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider, Josh Tobin, Maciek Chociej, Peter Welinder, Vikash Kumar, and Wojciech Zaremba. Multi-goal reinforcement learning: Challenging robotics environments and request for research. *arXiv preprint arXiv:1802.09464*, 2018. URL https://arxiv.org/abs/1802.09464.

[45] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994. doi: 10.1002/9780470316887.

[46] Youcef Saad and Martin H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7 (3):856–869, 1986. doi: 10.1137/0907058.

[47] Aliaksei Sandryhaila and José MF Moura. Discrete signal processing on graphs. *IEEE transactions on signal processing*, 61(7):1644–1656, 2013.

[48] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016. URL https://arxiv.org/abs/1511.05952.

[49] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. doi: 10.1038/s41586-020-03051-4.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

[50] Guy Shani, David Heckerman, and Ronen I. Brafman. An MDP-based recommender system. *Journal of Machine Learning Research*, 6:1265–1295, 2005. URL https://jmlr.org/papers/v6/shani05a.html.

[51] Jack Sherman and Winifred J. Morrison. Adjustment of an inverse matrix corresponding to a change in one element of a given matrix. *Annals of Mathematical Statistics*, 21(1):124–127, 1950. doi: 10.1214/aoms/1177729893.

[52] Jonathan Richard Shewchuk. An introduction to the conjugate gradient method without the agonizing pain. Technical report, School of Computer Science, Carnegie Mellon University, 1994. URL https://www.cs.cmu.edu/~quake-papers/painless-conjugate-gradient.pdf.

[53] David I. Shuman, Sunil K. Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. The emerging field of signal processing on graphs. *IEEE Signal Processing Magazine*, 30(3): 83–98, 2013. doi: 10.1109/MSP.2012.2235192.

[54] David Silver, Richard S. Sutton, and Martin Müller. Sample-based learning and search with permanent and transient memories. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pages 968–975. ACM, 2008. doi: 10.1145/1390156.1390280.

[55] Alistair Sinclair. Improved bounds for mixing rates of markov chains and multicommodity flow. *Combinatorics, Probability and Computing*, 1(4):351–370, 1992.

[56] Kimberly L. Stachenfeld, Matthew M. Botvinick, and Samuel J. Gershman. The hippocampus as a predictive map. *Nature Neuroscience*, 20(11):1643–1653, 2017. doi: 10.1038/nn.4650.

[57] Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning (ICML)*, pages 216–224. Morgan Kaufmann, 1990.

[58] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018. URL http://incompleteideas.net/book/the-book-2nd.html.

[59] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2): 181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1.

[60] John N. Tsitsiklis. Asynchronous stochastic approximation and q-learning. *Machine Learning*, 16(3):185–202, 1994.

[61] Harm Van Seijen and Rich Sutton. Planning by prioritized sweeping with small backups. In Sanjoy Dasgupta and David McAllester, editors, *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 361–369, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR. URL https://proceedings.mlr.press/v28/vanseijen13.html.

[62] Kaixin Wang, Kuangqi Zhou, Qixin Zhang, Jie Shao, Bryan Hooi, and Jiashi Feng. Towards better Laplacian representation in reinforcement learning with generalized graph drawing. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 10946–10957. PMLR, 2021. URL https://arxiv.org/abs/2107.05545.

[63] Wikipedia contributors. Laplacian matrix — Wikipedia, The Free Encyclopedia, 2026. URL https://en.wikipedia.org/wiki/Laplacian_matrix.

[64] David Wingate and Kevin D. Seppi. Prioritization methods for accelerating MDP solvers. *Journal of Machine Learning Research*, 6:851–881, 2005. URL https://jmlr.org/papers/v6/wingate05a.html.

[65] Wolfgang Woess. *Random walks on infinite graphs and groups*, volume 138. Cambridge university press, 2000.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

[66] Lixin Zou, Long Xia, Zhuoye Ding, Jiaxing Song, Weidong Liu, and Dawei Yin. Reinforcement learning to optimize long-term user engagement in recommender systems. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2810–2818, 2019. doi: 10.1145/3292500.3330668. URL https://dl.acm.org/doi/10.1145/3292500.3330668.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Appendix of "Graph Topology Augmentation for Prioritized Sweeping in Non-stationary Reinforcement Learning"

Table of Contents

Appendix A. Proofs of Theoretical Results 15

Appendix B. Algorithms and Implementation Details 24

Appendix C. Intuitions, Design Decisions, Limitations and Future Directions 30

Appendix D. Environments, Hyperparameters, and Experimental Protocol 33

Appendix E. Miscellaneous Supplement 38

Appendix A. Proofs of Theoretical Results

A.1 Analytical assumptions

First we address some stronger behaviorial assumptions or workarounds applied to ensure robustness throughout our analytics.

Assumption 1 (Communicating MDP)**.** *The MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ has finite state and action spaces and is* communicating*: for any two states $s, s' \in \mathcal{S}$, there exists a stationary policy under which s' is reachable from s with positive probability. Equivalently, every ε-soft stationary policy induces an irreducible transition matrix on $\mathcal{S}$. Since the Boltzmann policy design in 4.2 assigns nonzero probability to every action, the induced transition matrix P_t is* ***irreducible*** *for all t.*

Assumption 2 (Bounded reward signal)**.** *Rewards may be stochastic and may change exogenously over time, but their absolute value do not exceed $\Delta_{\max}$. This keeps residuals, value targets, and topology scores finite.*

Assumption 3 (Ergodic reward shifts)**.** *The sequence $\{(s_{r,t},\, \Delta_{r,t})\}_{t\geq 1}$ is* ***stationary-ergodic*** *under a measure μ_R, with mean shift amplitude $\bar{\Delta}_r \triangleq \mathbb{E}_{\mu_R}[\Delta_{r,t}] > 0$ and cumulative total variation $B_N \triangleq \sum_{t=1}^{N} \Delta_{r,t} = N\bar{\Delta}_r(1 + o(1))$ a.s.*

Assumption 4 (Slow topology drift for cumulative tracking)**.** *For the cumulative-tracking analysis, there exist a reference kernel $\bar{P}$ and a constant $\varepsilon_{\text{pol}} \geq 0$ such that throughout the analysis window,*

$$\|P_t - \bar{P}\|_\infty \leq \varepsilon_{\text{pol}}.$$

This is the frozen-kernel / slow-drift envelope used to compare the evolving process against a fixed-kernel ergodic reference.

Assumption 5 (Frozen-kernel positive local advantage)**.** *Under the frozen reference kernel $\bar{P}$ from Assumption 4, the reward-shift process has positive probability of placing strictly above-cutoff topology mass inside the K-neighborhood of the shift site:*

$$\mu_R\big(\exists s \in \mathcal{B}_K(s_r) : \tau(\bar{P}, \zeta)(s) > \tau_{(q+1)}(\bar{P}, \zeta)\big) > 0.$$

Corollary 1 (Well-definedness of the kernel P_t)**.** *The existence of a unique stationary distribution μ_t^{PS} requires the policy-induced kernel P_t to be irreducible and aperiodic. When this condition is not met exactly, it can be enforced by adding an auxiliary bridge edge with arbitrarily small probability $\varepsilon > 0$. Likewise, the Green's-function expansion in (5.2.1) is simplest when P_t is diagonalizable, which can be enforced by an arbitrarily small generic perturbation of one transition entry.*

A.2 Supporting Lemmas

Lemma A.2.1 (Invertible regularized Laplacian)**.** *The shifted systems $L_t^{out} + \eta I$ and $L_t^{in} + \eta I$ are invertible for every $\eta > 0$.*

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Proof. Since P_t is row-stochastic, $\sum_{j\neq i} p_{ij} = 1 - p_{ii}$. Hence $L_t^{\text{out}} + \eta I = (1+\eta)I - P_t$ is strictly row diagonally dominant, with diagonal gap $\eta > 0$.

Similarly, for $c_i = (P_t^\top \mathbf{1})_i$, the matrix $L_t^{\text{in}} + \eta I = D_t^{\text{in}} + \eta I - P_t^\top$ has diagonal gap

$$(c_i + \eta - p_{ii}) - (c_i - p_{ii}) = \eta > 0.$$

Both matrices are therefore nonsingular by the Levy–Desplanques theorem. □

Lemma A.2.2 (Resolvent norm bounds)**.** $\left\|(L_t^{out} + \eta I)^{-1}\right\|_\infty \leq 1/\eta$ *and* $\left\|(L_t^{in} + \eta I)^{-1}\right\|_\infty \leq 1/\eta$.

Proof. Both shifted matrices have the same useful structure. Let

$$A_t \in \{L_t^{out} + \eta I,\ L_t^{in} + \eta I\}.$$

From the row-surplus computation in Lemma A.2.1, A_t is a strictly row diagonally dominant Z-matrix with surplus η. Hence A_t is an invertible M-matrix, so $A_t^{-1} \geq 0$ entrywise.

Moreover, both directed Laplacians annihilate the all-ones vector:

$$L_t^{out}\mathbf{1} = 0, \qquad L_t^{in}\mathbf{1} = D_t^{in}\mathbf{1} - P_t^\top \mathbf{1} = 0.$$

Therefore

$$A_t\mathbf{1} = \eta\mathbf{1}, \qquad A_t^{-1}\mathbf{1} = \frac{1}{\eta}\mathbf{1}.$$

Since A_t^{-1} is entrywise nonnegative, its ℓ_∞ operator norm is the maximum row sum:

$$\left\|A_t^{-1}\right\|_\infty = \left\|A_t^{-1}\mathbf{1}\right\|_\infty = \frac{1}{\eta}.$$

Applying this to $A_t = L_t^{out} + \eta I$ and $A_t = L_t^{in} + \eta I$ gives the two desired bounds. □

Lemma A.2.3 (Geometric discount horizon)**.** *Let P_t be row-stochastic, $\gamma \in (0,1)$, and $\eta^\star = (1-\gamma)/\gamma$. Then*

$$(L_t^{\text{out}} + \eta^\star I)^{-1} = \gamma(I - \gamma P_t)^{-1} = \gamma \sum_{k\geq 0} (\gamma P_t)^k.$$

Thus, for any $z_t \geq 0$,

$$\left((L_t^{\text{out}} + \eta^\star I)^{-1} z_t\right)(s) = \gamma\, \mathbb{E}_{P_t}\left[\sum_{k\geq 0} \gamma^k z_t(X_k) \mid X_0 = s\right].$$

Proof. Since $L_t^{\text{out}} = I - P_t$,

$$L_t^{\text{out}} + \eta^\star I = \frac{1}{\gamma}(I - \gamma P_t).$$

The inverse identity follows. Since $\|\gamma P_t\|_\infty < 1$, the Neumann series applies. Finally, $(P_t^k z_t)(s) = \mathbb{E}_{P_t}[z_t(X_k) \mid X_0 = s]$. □

Corollary 2 (Exact topology-priority sorter)**.** *At $\eta = \eta^\star$, taking $z_t = |\xi_t|$ in Lemma A.2.3 gives*

$$\tau_t(s) = \left((L_t^{\text{out}} + \eta^\star I)^{-1}|\xi_t|\right)(s) = \gamma\, \mathbb{E}_{P_t}\left[\sum_{k\geq 0} \gamma^k |\xi_t(X_k)| \mid X_0 = s\right].$$

Hence the additive score τ_t ranks states by expected discounted future Bellman residual, reflecting the proactive amount of future Bellman updating that will be applied through the backup chain.

Lemma A.2.4. *The Bellman optimality operator $\mathcal{T}V(s) = r_t(s) + \gamma \max_a \sum_{s'} p(s'|s,a)V(s')$ is a γ-contraction in $\|\cdot\|_\infty$, independent of the topology policy π_t^{PS}.*

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Proof. Fix $V, V' : \mathcal{S} \to \mathbb{R}$ and a state $s \in \mathcal{S}$. Let

$$a_s \in \arg\max_a \sum_{s'} p(s'|s,a)V(s'), \qquad a'_s \in \arg\max_a \sum_{s'} p(s'|s,a)V'(s').$$

Using a_s as a feasible action for the maximization that defines $\mathcal{T}V'(s)$,

$$\begin{aligned}(\mathcal{T}V)(s) - (\mathcal{T}V')(s) &= \gamma \max_a \sum_{s'} p(s'|s,a)V(s') - \gamma \max_a \sum_{s'} p(s'|s,a)V'(s') \\ &\le \gamma \sum_{s'} p(s'|s,a_s)\big(V(s') - V'(s')\big) \\ &\le \gamma \left\|V - V'\right\|_\infty \sum_{s'} p(s'|s,a_s) = \gamma \left\|V - V'\right\|_\infty .\end{aligned}$$

Exchanging the roles of V and V' gives

$$(\mathcal{T}V')(s) - (\mathcal{T}V)(s) \le \gamma \left\|V - V'\right\|_\infty .$$

Hence

$$|(\mathcal{T}V)(s) - (\mathcal{T}V')(s)| \le \gamma \left\|V - V'\right\|_\infty .$$

Taking the supremum over $s \in \mathcal{S}$ yields

$$\left\|\mathcal{T}V - \mathcal{T}V'\right\|_\infty \le \gamma \left\|V - V'\right\|_\infty .$$

The formula for $\mathcal{T}$ depends only on p and r_t, so this contraction property is independent of the topology policy π_t^{PS}. □

A.3 Proofs of Theoretical Results

Proof of Lemma 5.1.1

Proof. Let

$$E_k \triangleq \left\|V_k - V^*\right\|_\infty .$$

At backup step k, only the selected state s_k is updated:

$$V_{k+1}(s_k) = (\mathcal{T}V_k)(s_k), \qquad V_{k+1}(s) = V_k(s) \text{ for } s \ne s_k.$$

Since $V^* = \mathcal{T}V^*$ and $\mathcal{T}$ is a γ-contraction by Lemma A.2.4,

$$|V_{k+1}(s_k) - V^*(s_k)| = |(\mathcal{T}V_k)(s_k) - (\mathcal{T}V^*)(s_k)| \le \gamma E_k.$$

For every $s \ne s_k$, the error is unchanged:

$$|V_{k+1}(s) - V^*(s)| = |V_k(s) - V^*(s)| \le E_k.$$

Hence $E_{k+1} \le E_k$ for every k, so the error sequence is nonincreasing.

Now define a sequence of cover times. Let $N_0 = 0$. Since every state is selected infinitely often and $\mathcal{S}$ is finite, there exists a finite time $N_1 > N_0$ such that every state appears at least once among

$$s_{N_0}, s_{N_0+1}, \ldots, s_{N_1-1}.$$

Inductively, once N_m is defined, there exists $N_{m+1} > N_m$ such that every state appears at least once among

$$s_{N_m}, s_{N_m+1}, \ldots, s_{N_{m+1}-1}.$$

Fix $m \ge 0$ and a state $s \in \mathcal{S}$. Let $j_s \in \{N_m, \ldots, N_{m+1} - 1\}$ be the last time in this block at which state s is updated. Immediately after that update,

$$|V_{j_s+1}(s) - V^*(s)| \le \gamma E_{j_s} \le \gamma E_{N_m},$$

because E_k is nonincreasing. Since s is not updated again between times $j_s + 1$ and N_{m+1}, its value stays fixed on that interval, so

$$\left|V_{N_{m+1}}(s) - V^*(s)\right| = |V_{j_s+1}(s) - V^*(s)| \le \gamma E_{N_m}.$$

This bound holds for every state s, therefore

$$E_{N_{m+1}} \le \gamma E_{N_m}.$$

Iterating gives

$$E_{N_m} \le \gamma^m E_0 \xrightarrow[m\to\infty]{} 0.$$

Finally, if $N_m \le k < N_{m+1}$, then $E_k \le E_{N_m}$ because the error sequence is nonincreasing. Hence $E_k \to 0$ as $k \to \infty$, i.e.

$$V_k \to V^* \quad \text{in } \left\|\cdot\right\|_\infty .$$

□

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Proof of Proposition 5.1

Proof. Assume that no further reward shifts occur, so the Bellman operator $\mathcal{T}$ is fixed.

Queue mechanism We restate the notations in B.4.1. Let

$$\xi_t(s) \triangleq (\mathcal{T}V_t)(s) - V_t(s), \qquad \mathcal{I}_t \triangleq \{s \in \mathcal{S} : |\xi_t(s)| > 0\}.$$

For convenience, $\mathcal{I}_t$ is asserted to be non-empty, as Assumption 1 and 4.2 ensure infinite exploration.

The implementation maintains the residual queue exactly over the nonzero reactive residuals,

$$\mathrm{Keys}(\mathcal{Q}_\xi(t)) = \mathcal{I}_t,$$

and a separate topology queue $\mathcal{Q}_\tau(t)$. Backups are selected by the combined priority

$$s_t \in \arg\max_{s \in \mathcal{I}_t} \{|\xi_t(s)| + \beta_t \tau_t(s)\}.$$

The topology score is computed from the full residual field through the outgoing and incoming directed Laplacian resolvents:

$$\mathbf{f}_t^{out} \triangleq (L_t^{out} + \eta^* I)^{-1}|\boldsymbol{\xi}_t|, \qquad \mathbf{f}_t^{in} \triangleq (L_t^{in} + \eta^* I)^{-1}|\boldsymbol{\xi}_t|.$$

The realized topology signal is the in–out mixture

$$\tau_t(s) = (1 - \alpha_t)|f_t^{out}(s)| + \alpha_t |f_t^{in}(s)|, \qquad \alpha_t \in [0, 1]. \tag{1}$$

Topology signal bound By Assumption 2, the reward signal is uniformly bounded; hence the Bellman targets and residuals remain finite. Moreover, by Lemma A.2.2,

$$\left\|(L_t^{out} + \eta^* I)^{-1}\right\|_\infty \le \frac{1}{\eta^*}, \qquad \left\|(L_t^{in} + \eta^* I)^{-1}\right\|_\infty \le \frac{1}{\eta^*}.$$

Therefore,

$$0 \le \tau_t(s) \le \frac{1}{\eta^*}\|\boldsymbol{\xi}_t\|_\infty < \infty. \tag{2}$$

The mixing-time modulation is

$$\beta_t = \beta_1(1 - \mathrm{SLEM}(P_t) + \beta_2).$$

Since $\mathrm{SLEM}(P_t) \le 1$ also gives $1 - \mathrm{SLEM} + \beta_2 \ge \beta_2$, we derive the tighter lower bound $\beta_t \ge \beta_1\beta_2 > 0$, which ensures $\beta_t\tau_t(s) \ge 0$ for every s. Combining this with (2) gives

$$0 \le \beta_t\tau_t(s) \le \frac{B_\beta}{\eta^*}\|\boldsymbol{\xi}_t\|_\infty \triangleq C_\tau\|\boldsymbol{\xi}_t\|_\infty. \tag{3}$$

Infinite-visit property. Let

$$F \triangleq \{s \in \mathcal{S} : s \text{ is selected only finitely often}\}, \qquad U \triangleq \mathcal{S} \setminus F.$$

If $F = \varnothing$, every state is selected infinitely often and Lemma 5.1.1 gives convergence directly.

Suppose $F \neq \varnothing$. Choose N such that no state in F is selected after time N. Then

$$V_t(f) = V_N(f), \qquad \forall f \in F,\ t \ge N.$$

However, the residuals of states in F may still change because their Bellman targets depend on the values of states in U.

Regard F as fixed boundary data and define the restricted Bellman operator on U by

$$(\mathcal{T}_F W)(u) \triangleq (\mathcal{T}V)(u), \qquad u \in U,$$

where $V = W \oplus V_N|_F$. Updates to $u \in U$ see the frozen boundary values $V_N|_F$, so $\mathcal{T}_F$ acts as a fixed γ-contraction on $\mathbb{R}^U$ for all $t \ge N$. Every state in U is selected infinitely often by definition, so asynchronous value iteration on this restricted system converges by Lemma 5.1.1:

$$\max_{u \in U} |\xi_t(u)| \to 0.$$

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Now suppose, toward a contradiction, that the finitely-selected set carries persistent residual:

$$\limsup_{t\to\infty} \max_{f\in F} |\xi_t(f)| = \delta > 0.$$

Choose

$$\varepsilon < \frac{(1-C_\tau)\delta}{4}.$$

Then there are arbitrarily large times $t \geq N$ such that

$$M_F(t) \triangleq \max_{f\in F} |\xi_t(f)| \geq \frac{\delta}{2}, \qquad M_U(t) \triangleq \max_{u\in U} |\xi_t(u)| \varepsilon.$$

Let $f_t \in \arg\max_{f\in F} |\xi_t(f)|$, so $f_t \in \mathcal{I}_t$. Since $M_F(t) \geq \delta/2 > \varepsilon > M_U(t)$, the F-states dominate the sup-norm: $\|\boldsymbol{\xi}_t\|_\infty = M_F(t)$. Therefore, for every $u \in U$, (3) gives

$$\begin{aligned} |\xi_t(u)| + \beta_t \tau_t(u) &\leq M_U(t) + C_\tau M_F(t) \\ &\varepsilon + C_\tau M_F(t) \\ &M_F(t) \\ &= |\xi_t(f_t)| \\ &\leq |\xi_t(f_t)| + \beta_t \tau_t(f_t), \end{aligned}$$

where the third line uses $\varepsilon < (1-C_\tau)M_F(t)$, which holds because $M_F(t) \geq \delta/2$ and $\varepsilon < (1-C_\tau)\delta/4$. Hence no state in U attains the $\arg\max$ of the combined priority over $\mathcal{I}_t$. Because $f_t \in F \cap \mathcal{I}_t$, the feasible set contains at least one F-state, so the selected backup s_t must lie in $F \cap \mathcal{I}_t \subseteq F$ — whether $s_t = f_t$ or another F-state reached by random tie-breaking. This contradicts the choice of N, which stipulates that no state in F is selected after time N. Therefore,

$$\limsup_{t\to\infty} \max_{f\in F} |\xi_t(f)| = 0.$$

Together with $\max_{u\in U} |\xi_t(u)| \to 0$, this gives

$$\|\boldsymbol{\xi}_t\|_\infty \to 0.$$

where the hypothesis $\beta_1(1+\beta_2) < \eta^*$ gives $C_\tau < 1$.

Convergence to V^*. Using $V^* = \mathcal{T}V^*$ and the γ-contractivity of $\mathcal{T}$,

$$\begin{aligned} \|V_t - V^*\|_\infty &\leq \|V_t - \mathcal{T}V_t\|_\infty + \|\mathcal{T}V_t - \mathcal{T}V^*\|_\infty \\ &\leq \|\boldsymbol{\xi}_t\|_\infty + \gamma\|V_t - V^*\|_\infty. \end{aligned}$$

Rearranging,

$$(1-\gamma)\|V_t - V^*\|_\infty \leq \|\boldsymbol{\xi}_t\|_\infty \to 0.$$

Therefore GTA-PS$^\beta$ converges deterministically to V^* in the no-shift regime. □

Proof of Lemma 5.2.1

Proof. The biorthogonal decomposition gives

$$P_t = \sum_k \lambda_k v_k u_k^\top, \qquad u_j^\top v_k = \delta_{jk}.$$

Let

$$M_t \triangleq L_t^{out} + \eta I = (1+\eta)I - P_t.$$

For each eigenpair (λ_k, v_k) of P_t,

$$M_t v_k = \big((1+\eta)I - P_t\big)v_k = (1+\eta-\lambda_k)v_k = (1-\lambda_k+\eta)v_k.$$

Since P_t is stochastic, every eigenvalue satisfies $|\lambda_k| \leq 1$. Therefore

$$\mathrm{Re}(1-\lambda_k+\eta) = 1 - \mathrm{Re}(\lambda_k) + \eta \geq \eta > 0,$$

so $1-\lambda_k+\eta \neq 0$ for every k. Thus M_t is invertible on every eigendirection and

$$M_t^{-1} v_k = \frac{1}{1-\lambda_k+\eta} v_k.$$

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Because $\{v_k, u_k\}$ form a biorthogonal basis, every vector admits the expansion

$$\xi_t = \sum_k (u_k^\top \xi_t)\, v_k.$$

Applying M_t^{-1} termwise gives

$$\begin{aligned}\mathbf{f}_t^{out} &= M_t^{-1}\xi_t \\ &= \sum_k (u_k^\top \xi_t)\, M_t^{-1} v_k \\ &= \sum_k \frac{u_k^\top \xi_t}{1-\lambda_k+\eta}\, v_k.\end{aligned}$$

This is the claimed spectral expansion.

For the amplification bound, each modal coefficient is multiplied by $(1-\lambda_k+\eta)^{-1}$, and

$$|1-\lambda_k+\eta| \geq \mathrm{Re}(1-\lambda_k+\eta) \geq \eta.$$

Hence

$$\frac{1}{|1-\lambda_k+\eta|} \leq \frac{1}{\eta},$$

so no mode is amplified by more than $1/\eta$. The same argument applies to periodic chains: even if a nontrivial eigenvalue lies on the unit circle, the positive regularization parameter η keeps the denominator away from zero.

The final sentence of the theorem now follows directly from the spectral ordering: $k=1$ is the Perron mode, while $k=2$ is the first non-Perron mode associated with the SLEM. □

Proof for Proposition 5.2 Ở đây có lỗi psi(s) = gamma*G_P_t^*(s,s_r)

Proof. Set $P_t^\star \triangleq \gamma P_t$. By the Neumann series, $\psi(s) = G_{P_t^\star}(s, s_r)$. Positivity $\psi(s) > 0$ follows from Assumption 1: for every s there exists $m \geq 1$ with $P_t^m(s, s_r) > 0$, so $G_{P_t^\star}(s, s_r) \geq \gamma^m P_t^m(s, s_r) > 0$. The identity $\eta^* = (1-\gamma)/\gamma$ gives $L_t^{out} + \eta^* I = \frac{1}{\gamma}(I - \gamma P_t)$, hence $(L_t^{out} + \eta^* I)^{-1} = \gamma(I - \gamma P_t)^{-1}$, confirming $\psi(s) = G^{out}_{\eta^*,t}(s, s_r)$.

Harmonicity. The resolvent identity $Q(I-Q)^{-1} = (I-Q)^{-1} - I$ with $Q = \gamma P_t$ gives

$$(P_t^\star \psi)(s) = \gamma\big[\gamma P_t (I-\gamma P_t)^{-1}\big]_{s,s_r} = \gamma\big[(I-\gamma P_t)^{-1} - I\big]_{s,s_r} = \psi(s) - \gamma\delta_{s,s_r}, \tag{4}$$

so $P_t^\star \psi = \psi$ pointwise on $\mathcal{S} \setminus \{s_r\}$. Hiện tượng hấp thụ này khẳng định toán học rằng thế tô-pô ψ không phân phối xác suất đi vòng vo vô tận, mà nó dồn toàn bộ vector hướng dòng chảy (gradient) đổ dồn về đích đến duy nhất là s_r.

Row sums of $\tilde{P}_t$. From (4),

$$\sum_{s'} \tilde{P}_t(s, s') = \frac{(P_t^\star \psi)(s)}{\psi(s)} = 1 - \frac{\gamma\, \delta_{s,s_r}}{\psi(s)}.$$

Hence $\tilde{P}_t$ is stochastic on $\mathcal{S} \setminus \{s_r\}$ and has row-sum defect $\gamma/\psi(s_r)$ at s_r, which is the standard Doob absorption at the target site [65, Thm. 7.2].

Telescoping and importance sampling. For any $P_t^\star$-positive path $(X_0, \ldots, X_k)$,

$$\frac{d\tilde{\mathbb{P}}}{d\mathbb{P}}\big[(X_0, \ldots, X_k)\big] \;=\; \prod_{j=0}^{k-1} \frac{\tilde{P}_t(X_j, X_{j+1})}{P_t^\star(X_j, X_{j+1})} \;=\; \prod_{j=0}^{k-1} \frac{\psi(X_{j+1})}{\psi(X_j)} \;=\; \frac{\psi(X_k)}{\psi(X_0)}.$$

The initial factor $\psi(X_0)^{-1}$ is the exact importance weight for the path-measure change. Since $\psi(s) = G_{P_t^\star}(s, s_r)$ equals the $\tilde{P}_t$-expected discounted hitting mass at s_r [65, Prop. 7.4], ranking by $\psi(s)$ concentrates backups on the states with maximal $\tilde{P}_t$-path weight toward the reward-shift site. □

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Proof for Proposition 5.3

Proof. (i). At episode start after a single-site reward shift,

$$\xi_t = \Delta_r e_{s_r}.$$

By Cor. 2,

$$\tau_t = \gamma(I - \gamma P_t)^{-1}\xi_t = \gamma\Delta_r \sum_{k=0}^{\infty}(\gamma P_t)^k e_{s_r}.$$

Evaluating the s-th component gives

$$\tau_t(s) = \gamma\Delta_r \sum_{k=0}^{\infty}\gamma^k [P_t^k]_{s,s_r}.$$

Every term in this series is nonnegative. Because P_t is finite and irreducible, for every pair (s, s_r) there exists some $k = k(s, s_r)$ such that $[P_t^k]_{s,s_r} > 0$. Therefore at least one term in the series is strictly positive, and hence

$$\tau_t(s) > 0 \qquad \text{for every } s \in \mathcal{S}.$$

This proves dense support.

(ii). Fix $s \in \mathcal{B}_K$, by single-site reward shift assumption, for $s \in \mathcal{B}_K$

$$h^*(s; s_r) > K.$$

By definition of the backward predecessor distance, there is no path

$$s = s_0, s_1, \ldots, s_\ell = s_r$$

with $\ell \le K$ and $P_t(s_{i+1}, s_i) > 0$ for all i. Reactive PS propagates nonzero priority from the shifted reward state s_r only through predecessor pushes, one predecessor edge per backup. Therefore after at most K reactive backups, the residual wavefront can reach only states at backward predecessor distance at most K from s_r. Since s lies outside that K-step predecessor set, its reactive PS priority remains zero:

$$p_{\mathrm{PS}}(s) = 0.$$

By part (i), the GTA-PS topology signal still satisfies $\tau_t(s) > 0$. Hence GTA-PS assigns nonzero topology priority to every blocked state while reactive PS does not. This proves part (ii). □

Remark 1 (Blocked-set non-emptiness)**.** *Under Assumption 4a, define the backward directed eccentricity of the shifted reward state by*

$$D^*(s_r) \triangleq \max_{s\in\mathcal{S}} h^*(s; s_r).$$

Then the blocked set $\mathcal{B}_K = \{s : h^*(s; s_r) > K\}$ is non-empty whenever $K < D^*(s_r)$. In bottlenecked environments, $D^*(s_r)$ is large relative to the backup budget, so blocked states exist for the budgets where GTA-PS is intended to help.

For periodic chains, the same structural claim still holds because it depends only on the support graph. Independently, the regularized resolvent remains well-defined because $\eta^\star = (1-\gamma)/\gamma > 0$.

Proof for Lemma 5.3.1 and Proposition 5.4

Proof. We separate the argument into the fixed-kernel ergodic average and the drift perturbation.

Step 1: fixed-kernel ergodic average. For a fixed stochastic kernel P, Assumption 3 makes the reward-shift descriptor $Z_t = (s_{r,t}, \Delta_{r,t})$, and hence $\zeta_t = \Delta_{r,t} e_{s_{r,t}}$, stationary-ergodic.

$$\zeta \longmapsto \mathrm{adv}_t(P, \zeta)$$

is measurable and integrable, then Birkhoff's ergodic theorem gives

$$\frac{1}{N}\sum_{t=1}^{N} \mathrm{adv}_t(P, \zeta_t) \xrightarrow[N\to\infty]{\text{a.s.}} \bar{A}(P) \triangleq \mathbb{E}_{\mu_R}[\mathrm{adv}_t(P, \zeta)].$$

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Because the state space is finite and $\tau(P,\zeta) \triangleq \gamma(I-\gamma P)^{-1}\zeta$ is obtained by a bounded linear map applied to the bounded random vector ζ, the functional $\mathrm{adv}(P,\zeta)$ is measurable and integrable.

The positive-frequency assumption $|\mathcal{B}_K^{\mathrm{sel},t}| \geq 1$ implies that

$$\mu_R\big(\mathrm{adv}_t(P,\zeta_t) > 0\big) > 0$$

under the frozen-kernel process. Since each summand in the definition of adv is nonnegative, and $\mathbb{E}[X] \geq \mathbb{E}[X\,\mathbf{1}_{X>0}] > 0$ whenever $X \geq 0$ and $P(X>0) > 0$, it follows that

$$\bar{A}(P) > 0.$$

Step 2: decomposition around a reference kernel. By Assumption 4, there exists a frozen reference kernel $\bar{P}$ such that $\|P_t - \bar{P}\|_\infty \leq \varepsilon_{\mathrm{pol}}$ for all t. Then

$$\mathrm{adv}_t(P_t,\zeta_t) = \mathrm{adv}_t(\bar{P},\zeta_t) + \Big(\mathrm{adv}_t(P_t,\zeta_t) - \mathrm{adv}_t(\bar{P},\zeta_t)\Big).$$

Summing over t and dividing by N yields

$$\frac{1}{N}\sum_{t=1}^{N}\mathrm{adv}_t(P_t,\zeta_t) = \frac{1}{N}\sum_{t=1}^{N}\mathrm{adv}_t(\bar{P},\zeta_t) + \frac{1}{N}\sum_{t=1}^{N}\Big(\mathrm{adv}_t(P_t,\zeta_t) - \mathrm{adv}_t(\bar{P},\zeta_t)\Big).$$

The first term converges almost surely to $\bar{A}(\bar{P})$ by Step 1; write the convergence error as $e_N \triangleq \frac{1}{N}\sum_t \mathrm{adv}_t(\bar{P},\zeta_t) - \bar{A}(\bar{P}) = o(1)$ a.s.

Step 3: drift error bound. For two kernels P, P', the resolvent identity gives

$$(I-\gamma P)^{-1} - (I-\gamma P')^{-1} = (I-\gamma P)^{-1}\,\gamma(P-P')\,(I-\gamma P')^{-1},$$

and therefore, recalling $\tau(P,\zeta) = \gamma(I-\gamma P)^{-1}\zeta$,

$$\tau(P,\zeta) - \tau(P',\zeta) = \gamma^2(I-\gamma P)^{-1}(P-P')(I-\gamma P')^{-1}\zeta.$$

Since $\|P\|_\infty = 1$ implies $\|(I-\gamma P)^{-1}\|_\infty \leq 1/(1-\gamma)$ via the Neumann series,

$$\|\tau(P,\zeta) - \tau(P',\zeta)\|_\infty \leq \frac{\gamma^2}{(1-\gamma)^2}\,\|P-P'\|_\infty\,\|\zeta\|_\infty\,.$$

The order-statistic map $x \mapsto x_{(q+1)}$ is 1-Lipschitz in $\|\cdot\|_\infty$ (since $|x_{(k)} - y_{(k)}| \leq \|x-y\|_\infty$ for any rank k, by a standard sorting argument), so

$$\big|\tau_{(q+1)}(P,\zeta) - \tau_{(q+1)}(P',\zeta)\big| \leq \|\tau(P,\zeta) - \tau(P',\zeta)\|_\infty\,.$$

Because $x \mapsto (x)_+$ is also 1-Lipschitz, and recalling $\mathrm{adv}(P,\zeta) = \frac{1}{\gamma}\sum_{s\in\mathcal{B}_K(s_r)}\big(\tau(P,\zeta)(s) - \tau_{(q+1)}(P,\zeta)\big)_+$,

$$\begin{aligned}
&|\mathrm{adv}(P,\zeta) - \mathrm{adv}(P',\zeta)| \\
&\leq \frac{1}{\gamma}\sum_{s\in\mathcal{B}_K(s_r)}\Big(|\tau(P,\zeta)(s) - \tau(P',\zeta)(s)| + \big|\tau_{(q+1)}(P,\zeta) - \tau_{(q+1)}(P',\zeta)\big|\Big) \\
&\leq \frac{2|\mathcal{B}_K(s_r)|}{\gamma}\,\|\tau(P,\zeta) - \tau(P',\zeta)\|_\infty \\
&\leq \frac{2|\mathcal{B}_K(s_r)|\,\gamma}{(1-\gamma)^2}\,\|\zeta\|_\infty\,\|P-P'\|_\infty\,.
\end{aligned}$$

Since $\|\zeta_t\|_\infty = \Delta_{r,t} \leq \Delta_{\max}$ (the maximum absolute reward perturbation across states at time t) and $|\mathcal{B}_K(s_r)| \leq |\mathcal{S}|$, the functional is uniformly Lipschitz in P with constant

$$L \triangleq \frac{2|\mathcal{S}|\gamma\Delta_{\max}}{(1-\gamma)^2}.$$

Therefore

$$\big|\mathrm{adv}_t(P_t,\zeta_t) - \mathrm{adv}_t(\bar{P},\zeta_t)\big| \leq L\,\big\|P_t - \bar{P}\big\|_\infty \leq L\varepsilon_{\mathrm{pol}}$$

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

throughout the drift envelope. Hence

$$\frac{1}{N}\sum_{t=1}^{N}\Big(\mathrm{adv}_t(P_t,\zeta_t)-\mathrm{adv}_t(\bar{P},\zeta_t)\Big)\geq -L\varepsilon_{\mathrm{pol}}.$$

Combining with Step 2 gives

$$\frac{1}{N}\sum_{t=1}^{N}\mathrm{adv}_t(P_t,\zeta_t)\geq \bar{A}(\bar{P})-L\varepsilon_{\mathrm{pol}}+e_N=\bar{A}(\bar{P})-L\varepsilon_{\mathrm{pol}}+o(1)\qquad\text{a.s.}$$

Renaming $\bar{A}(\bar{P})$ as $\bar{A}$ yields the displayed conclusion. □

Proof of Theorem 5.1

Proof. **Part (i): per-episode GTA-PS residual.** By Cor. 2,

$$\mathrm{err}_t^{\mathrm{GTA}}=\frac{\tau_{(q+1),t}}{\gamma}=\Phi_{(q+1),t}\leq\frac{\|\xi_t\|_\infty}{1-\gamma}\leq\frac{\Delta_{r,t}}{1-\gamma}.$$

The first equality is the definition of $\mathrm{err}_t^{\mathrm{GTA}}$. The second is the exact-sorter identity. For the first inequality,

$$\Phi_t=(I-\gamma P_t)^{-1}\xi_t=\sum_{k=0}^{\infty}(\gamma P_t)^k\xi_t,$$

so

$$\|\Phi_t\|_\infty\leq\sum_{k=0}^{\infty}\gamma^k\|\xi_t\|_\infty=\frac{1}{1-\gamma}\|\xi_t\|_\infty.$$

Since $\Phi_{(q+1),t}$ is one coordinate of Φ_t, this implies $\Phi_{(q+1),t}\leq\|\xi_t\|_\infty/(1-\gamma)$. At episode start, under the single-shift assumption: $\xi_t=\Delta_{r,t}e_{s_r}$. Hence $\|\xi_t\|_\infty=\Delta_{r,t}$ and the final inequality follows.

Part (ii): cumulative advantage reduction. By Prop. 5.3(ii), reactive PS leaves every blocked state $s^*\in\mathcal{B}_K^{\mathrm{sel},t}$ unresolved after the $q=K$ within-episode backups. By Cor. 2, its discounted future residual there is $\Phi_t(s^*)=\tau_t(s^*)/\gamma$. Under the conservative cutoff convention, GTA-PS reduces the unresolved residual on that state to the cutoff level $\tau_{(q+1),t}/\gamma$. Hence each selected blocked state contributes $(\tau_t(s^*)-\tau_{(q+1),t})/\gamma$ to the reactive-PS minus GTA-PS gap, and summing over all such states gives

$$\mathrm{TE}^{\mathrm{PS}}(N)-\mathrm{TE}^{\mathrm{GTA}}(N)\geq\sum_{t=1}^{N}\sum_{s^*\in\mathcal{B}_K^{\mathrm{sel},t}}\frac{\tau_t(s^*)-\tau_{(q+1),t}}{\gamma}.$$

The states $s^*\in\mathcal{B}_K^{\mathrm{sel},t}$ are precisely those whose τ_t-value exceeds the $(q+1)$-th order statistic $\tau_{(q+1),t}$, so each summand is nonnegative and the $(\cdot)_+$ in the definition of adv_t is tight; therefore the right-hand side equals $\sum_{t=1}^{N}\mathrm{adv}_t(P_t,\zeta_t)$. By Prop. 5.4, multiplying its a.s. bound $(1/N)\sum_t\mathrm{adv}_t\geq\bar{A}-L\varepsilon_{\mathrm{pol}}+o(1)$ through by N,

$$\sum_{t=1}^{N}\mathrm{adv}_t(P_t,\zeta_t)\geq N(\bar{A}-L\varepsilon_{\mathrm{pol}})+o(N)\qquad\text{a.s.}$$

This proves the theorem's displayed lower bound. If in addition $L\varepsilon_{\mathrm{pol}}<\bar{A}$, then the right-hand side tends to $+\infty$, so

$$\mathrm{TE}^{\mathrm{PS}}(N)-\mathrm{TE}^{\mathrm{GTA}}(N)\to+\infty$$

almost surely. □

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Appendix B. Algorithms and Implementation Details

B.1 Algorithmic settings for MDP observability

As to aid the integration of GTA-PS onto DP and Dyna, we can not ignore what each of them is built for in the first place.

DP can only be chosen when the full oracle distributional setting is exposed. The host agent's belief cannot exceed that of exact planning model, as every intel: the known transition kernel, the current reward distribution, and the ideal Bellman residual can be traced at ease. The host does not need exploratory action selection onto the MDP to collect data. The one actual committal process – being Policy Improvement – serves as the host step.

In the sample-based Dyna setting, because of 3, the host agent's belief must be empirical: it can only be formed through observed experience, by typically maintaining a learned model plus the current reward estimate. The available information is therefore local and noisy, best managed by one-step TD residuals rather than other more sample consuming ones. The host step is action execution under an exploratory move, as this leads for subsequent learning process.

Note. In Dyna, the *host policy* should be an ε-greedy one, so that new experience continues to update the sample model and not get stuck in stale knowledge.

B.2 Computational methods

B.2.1 Construction of L_t^{out}, L_t^{in}.

Given per-action matrices P^a and topology policy π_t^{PS}, construct $P_t = \sum_a \mathrm{diag}(\pi_t^{\mathrm{PS}}(\cdot, a))P^a$ in $O(|\mathcal{S}| \cdot |\mathcal{A}| \cdot \deg)$. Then $L_t^{out} = I - P_t$ and $L_t^{in} = D_t^{in} - P_t^\top$ where $D_t^{in} = \mathrm{diag}(P_t^\top \mathbf{1})$. Both are stored in CSR format.

B.2.2 Resolvent computation.

$L_t^{out} = I - P_t$ is singular (null vector $\mathbf{1}$). Adding ηI shifts all eigenvalues by η, so $\mathrm{Re}(\lambda_k(L_t^{out} + \eta I)) \geq \eta > 0$ for any ergodic chain. The inverse is thus well-defined without any symmetry requirement. The forward and backward potentials are residual diffusions:

$$\mathbf{f}_t^{out} = (L_t^{out} + \eta I)^{-1}\xi_t, \qquad \mathbf{f}_t^{in} = (L_t^{in} + \eta I)^{-1}\xi_t.$$

For small problems ($|\mathcal{S}| \leq 300$ in the reference implementation), the sparse systems are factored once with `scipy.sparse.linalg.splu`; repeated resolvent evaluations then reduce to triangular solves through `lu.solve(rhs)`. This is the path used by the tabular FourRooms and small GARNET experiments.

For larger problems, a realistic sparse iterative path is restarted GMRES with an ILU preconditioner [46, 38]. In SciPy notation, this is `spla.spilu(A.tocsc(), fill_factor=10, drop_tol=1e-4)` followed by `spla.gmres(A, rhs, M=M, restart=20, maxiter=200, atol=1e-8)`. If ILU fails because of a singular pivot, or if GMRES stalls, the implementation falls back to `spla.spsolve` or a fresh LU factorization.

Reward shifts are especially cheap in the LU regime because the operator is unchanged: if $\delta\mathbf{r}_t$ is the new reward perturbation, the update is $\mathbf{f}_t^{out} \leftarrow \mathbf{f}_t^{out} + (L_t^{out} + \eta I)^{-1}\delta\mathbf{r}_t$ and analogously for $\mathbf{f}_t^{in}$. A full column cache $g(\cdot, s)$ would reduce repeated reward-shift queries to $O(|\mathrm{supp}(\delta\mathbf{r})| \cdot |\mathcal{S}|)$, but at $O(|\mathcal{S}|^2)$ memory cost; the reference path instead reuses sparse factors and solves on demand.

B.2.3 SLEM computation via Arnoldi.

The SLEM is recomputed from the current topology kernel P_t using the Arnoldi process underlying `scipy.sparse.linalg.eigs`. Each matrix-vector product costs $O(\mathrm{supp}(P_t))$, so a k-vector Krylov run costs $O(k\,\mathrm{supp}(P_t) + k^2|\mathcal{S}|)$. In practice we use $k = 15$ and pass the dominant eigenvector from the previous recomputation as a warm start. The non-unit eigenvalue with the largest modulus is taken as the SLEM estimate. Recomputing every $T_{\mathrm{arn}} = \lceil\sqrt{|\mathcal{S}|}\rceil$ policy updates is usually sufficient; between those checkpoints, the scheduler reuses the last estimate.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 2 Resolvent Solve Path

$P_{\text{sp}} \leftarrow \texttt{sp.csr_array}(P_t)$
$A^{out} \leftarrow \texttt{sp.eye}(n, \texttt{format="csr"}) \cdot (1+\eta) - P_{\text{sp}}$
$A^{in} \leftarrow \texttt{in_laplacian}(P_t) + \texttt{sp.eye}(n, \texttt{format="csr"}) \cdot \eta$
if $n \leq 300$ **then**
    $\texttt{lu_out} \leftarrow \texttt{spla.splu}(A^{out}\texttt{.tocsc()})$
    $\texttt{lu_in} \leftarrow \texttt{spla.splu}(A^{in}\texttt{.tocsc()})$
    $\mathbf{f}_t^{out} \leftarrow \texttt{lu_out.solve}(\xi_t)$
end if
if $n > 300$ **then**
    $\texttt{ilu} \leftarrow \texttt{spla.spilu}(A^{\text{out}}\texttt{.tocsc()}, \texttt{fill_factor} = 10, \texttt{drop_tol} = 10^{-4})$
    $M \leftarrow \texttt{spla.LinearOperator}((n,n), \texttt{ilu.solve})$
    $\mathbf{f}_t^{\text{out}}, \texttt{info} \leftarrow \texttt{spla.gmres}(A^{\text{out}}, \xi_t, M, \texttt{restart} = 20, \texttt{maxiter} = 200, \texttt{atol} = 10^{-4})$
    if $\texttt{info} \neq 0$ **then**
        $\mathbf{f}_t^{\text{out}} \leftarrow \texttt{spla.spsolve}(A^{\text{out}}\texttt{.tocsc()}, \xi_t)$
    end if
end if
$\mathbf{f}_t^{in}$ is solved analogously with A^{in}

Algorithm 3 SLEM Update

$P_{\text{sp}} \leftarrow \texttt{sp.csr_array}(P_t)$; $P_{\text{sp}}\texttt{.eliminate_zeros()}$
$\lambda,\ V \leftarrow \texttt{spla.eigs}(P_{\text{sp}}, k = 16, \texttt{which="LM"}, v_0 = v_{\text{prev}})$
$m_i \leftarrow |\lambda_i|$ for all returned eigenvalues
discard the eigenvalue with $|\lambda_i - 1| \leq 10^{-4}$
$\lambda_2 \leftarrow \arg\max_i m_i$ over the remaining set
$\text{SLEM}(P_t) \leftarrow |\lambda_2|$
$v_{\text{prev}} \leftarrow \text{Re}(V[:,0])$ for the next warm start
recompute only if $t \bmod T_{\text{arn}} = 0$

B.2.4 Topology signal assembly.

Once ξ_t is available, the computational chain is: solve the forward and backward resolvents, mix them with the fixed coefficient α, filter the dense signal to a top-q_t frontier, and rekey the residual queue with the aggregate topology score. In the exact-model regime $\xi_t(s) = (\mathcal{T}V_t)(s) - V_t(s)$; in sample-based Dyna this can be replaced by the most recent TD(0) error. The reference implementation materializes the dense frontier with `np.argpartition` and keeps the active residual and topology-only candidates in separate heaps (`heapq.heapify`), which avoids sorting all states at every backup.

Algorithm 4 Topology Assembly

$\xi_t(s) \leftarrow (\mathcal{T}V_t)(s) - V_t(s)$
$\mathbf{f}_t^{\text{out}} \leftarrow \texttt{solve_out}(P_t, \xi_t))$; $\mathbf{f}_t^{\text{in}} \leftarrow \texttt{solve_in}(P_t, \xi_t)$
$\boldsymbol{\tau}_t \leftarrow (1-\alpha)|\mathbf{f}_t^{\text{out}}| + \alpha|\mathbf{f}_t^{\text{in}}|$
$\beta_t \leftarrow \beta$ or $\beta_1((1 - \text{SLEM}(P_t)) + \beta_2)$
$I_q \leftarrow \texttt{np.argpartition}(\boldsymbol{\tau}_t, n - q_t)[n - q_t :]$
$\texttt{Q}_\xi \leftarrow \texttt{heapq.heapify}\{(s, |\xi_t(s)| + \beta_t\tau_t(s)) : |\xi_t(s)| > 0\}$
$\texttt{H}_q \leftarrow \texttt{heapq.heapify}\{(s, \beta_t\tau_t(s)) : s \in I_q, \}$

B.2.5 Two-queue priority with top-q filtering.

Because $\boldsymbol{\tau}_t$ is dense, inserting it directly into the residual heap would crowd the queue. The implementation uses two persistent structures: $\mathcal{Q}_\xi$ over the active residual set and $\mathcal{Q}_\tau$ over all states keyed by $\tau_t(s)$. After each RefreshTau, a filtered frontier $\mathcal{F}_q = \text{TopQ}(\mathcal{Q}_\tau, q_t)$ is materialized (via argpartition, $O(|\mathcal{S}|)$) and stored as a small heap $\mathcal{H}_q$ keyed by the aggregate score $\xi[s] + \beta_t\tau[s]$. The backup candidate set is $\mathcal{C}_t = \text{Keys}(\mathcal{Q}_\xi) \cup \mathcal{F}_q$, with selection by $\arg\max_{s \in \mathcal{C}_t} \{|\xi_t(s)| + \beta_t\tau_t(s)\}$. Setting $q_t = |\mathcal{S}|$ recovers the unfiltered algorithm.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

B.2.6 Computation chain.

The safest way to read the deployed module is as a directed acyclic computation graph.

At a fixed host step, the topology-policy state determines the induced topology kernel and its two linear systems:

$$\pi_t^{\mathrm{PS}} \;\longrightarrow\; P_t \;\longrightarrow\; (L_t^{out}, L_t^{in}).$$

From that point the computation splits into three downstream branches. The spectral branch is

$$P_t \;\longrightarrow\; \mathrm{SLEM}(P_t) \;\longrightarrow\; \beta_t.$$

The dense topology branch is

$$(\xi_t, L_t^{out}, L_t^{in}) \;\longrightarrow\; (\mathbf{f}_t^{out}, \mathbf{f}_t^{in}) \;\longrightarrow\; \boldsymbol{\tau}_t \;\longrightarrow\; \mathcal{Q}_\tau \;\longrightarrow\; \mathcal{F}_q \;\longrightarrow\; \mathcal{H}_q.$$

The residual branch is simply

$$\xi_t \;\longrightarrow\; \mathcal{Q}_\xi.$$

These branches meet only at the comparison stage through the score $|\xi_t(s)| + \beta_t \tau_t(s)$ used by PopAndPrioritize to compare the head of $\mathcal{Q}_\xi$ against the head of $\mathcal{H}_q$. In particular, β_t does not enter the resolvent solves, $\boldsymbol{\tau}_t$ does not enter Arnoldi, and top-q_t filtering is applied only after the dense topology field has already been formed.

B.2.7 Per-step cost summary.

Procedure	Cost
GtaPolicyUpdate	$O(\lvert\mathcal{A}\rvert)$ per backed-up state
TopologyUpdate (per policy step)	$O(\lvert\mathcal{S}\rvert)$ (LU tri-solves + SM)
RefreshTau (per shift / lazy)	$O(\lvert\mathcal{S}\rvert + q_t)$
RewardUpdate (per shift)	$O(\lvert\mathcal{S}\rvert + q_t)$
PopAndPrioritize (per backup)	$O(\log \lvert\mathcal{S}\rvert + \log q_t)$
PushPredecessors (per backup)	$O(\lvert\mathrm{Pred}(s)\rvert \log \lvert\mathcal{S}\rvert)$
Arnoldi (every T_{arn})	$O(j\lvert\mathcal{S}\rvert \cdot \deg + j^2\lvert\mathcal{S}\rvert)$, $j = 15$

Dense refreshes (RefreshTau, Arnoldi) are triggered only at reward shifts and lazily every $T_\tau = |\mathcal{S}|$ backups; the amortized incremental cost per plan step is $O(|\mathcal{S}|)$ for the LU tri-solves and $O(|\mathcal{A}| \cdot \deg)$ for the host Bellman backup.

B.3 The unified Prioritized Sweeping interface

We should view any priority-based system as a *database* for the host. The host agent gains complete control over when entries are pushed, queried, refreshed, and removed during planning. The priority module should exposes a small set of stateful operations that the host can invoke inside DP or Dyna-style procedures, while preserving the same residual and topology bookkeeping across all variants. This makes the interface reusable across different replanning schedules, reward-update patterns, transition models and stopping rules. We do not specify how this should be implemented programmatically. However you can see the GTA-PS + DP/Dyna pseudocode in E.1 for the concrete host-side call sequence.

B.4 The GTA-PS module

B.4.1 Residual gating mechanism

GTA-PS precautiously mix topology as an ordering correction specifically over the residual support. At each planning step, define the residual-active set

$$\mathcal{I}_t \triangleq \{s \in \mathcal{S} : |\xi_t(s)| > 0\}.$$

If $\mathcal{I}_t \neq \varnothing$, the next backup state is selected only from $\mathcal{I}_t$:

$$s_t \in \arg\max_{s \in \mathcal{I}_t} \{|\xi_t(s)| + \beta_t \tau_t(s)\}.$$

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

and if not, exploratory action is triggered to collect more data.

Thus the topology term $\beta_t \tau_t(s)$ can reorder states that already have positive Bellman residual, but it cannot select a topology-only state while residual-active states exist.

This residual gate gives the anti-starvation arguments [9, 60], needed for the asynchronous contraction proof. Therefore GTA-PS preserves the standard infinite-residual visit condition used by asynchronous value-iteration convergence arguments.

B.4.2 Detailed pseudocode

The GTA-PS module exposes five procedures that any model-based host algorithm can call. The module maintains:

(i) the Bellman residual array $\xi[s]$ and a residual heap $\mathcal{Q}_\xi$ over the active set $\mathcal{I}_t = \{s : \xi[s] > 0\}$; $\mathcal{I}_t = \{s : \xi[s] > 0\}$ B.4.1;

(ii) the topology array $\tau[s]$, a dense heap $\mathcal{Q}_\tau$ over all states, and a filtered frontier $\mathcal{F}_q = \textsc{TopQ}(\mathcal{Q}_\tau, q_t)$ as a small heap $\mathcal{H}_q$ B.2.5;

(iii) topology state $(\mathbf{f}^{out}, \mathbf{f}^{in}, \mathrm{SLEM}, v_2, u_2)$;

(iv) the topology policy π_t^{PS} (Boltzmann over host advantages, temperature b).

The host algorithm (DP or Dyna-Q) calls PopAndPrioritize to select the next state for a Bellman backup, then calls PushPredecessors to propagate updated residuals to predecessor states. At each backed-up state the host also calls GtaPolicyUpdate (to refresh the temperature-softened Boltzmann policy row) followed by TopologyUpdate (to maintain $\mathbf{f}^{out}$, SLEM, and periodically $\mathbf{f}^{in}$ via the Sherman–Morrison and Arnoldi incremental machinery). After a reward shift the host calls RewardUpdate to diffuse the new residual field through both resolvents and rebuild priority queues.

B.5 Complexity analysis.

The complexity story splits cleanly into a local per-backup cost and three optional amortized corrections: a standalone dense τ refresh, a dense topology-operator rebuild, and an Arnoldi/SLEM sync. The local cost is paid on every host backup and comes from PopAndPrioritize, the Bellman or TD backup itself, PushPredecessors, and, in the online Sherman–Morrison regime, the row-local TopologyUpdate. The amortized terms are paid only when one of the dense refresh or spectral-sync events is triggered.

For bookkeeping, write

$$C_\tau \;=\; O\big(C_{\mathrm{pot}} + |\mathcal{S}| + q_t + |\mathcal{I}_t|\big)$$

for one RefreshTau call: the resolvent solves, dense mixing, top-q_t extraction, and queue rebuild/rekey. Let

$$C_{\mathrm{arn}} \;=\; O\big(j|\mathcal{S}| \cdot \deg + j^2|\mathcal{S}|\big)$$

for one Arnoldi recomputation with Krylov depth j, let C_{topo} denote the cost of one dense policy-dependent topology rebuild after batching row changes, and let

$$C_{\mathrm{shift}} \;=\; O\big(C_{\mathrm{res}} + C_\tau\big)$$

for one reward shift, since a shift recomputes the host residual field and then refreshes τ but does not change P_t or require a fresh Arnoldi pass. If a particular implementation couples some of these events, the corresponding costs simply merge; the split here is only to keep the accounting explicit.

The relevant granularity for the host planner is therefore one *plan step* or one backed-up state. For DP, the exact per-backup cost with GTA-PS is

$$C_{\mathrm{DP}}(s) = O\Big(|\mathcal{A}| \cdot \deg + |\mathrm{Pred}(s)| \log |\mathcal{S}| + \log |\mathcal{S}| + \log q_t + |\mathcal{S}|\Big),$$

where the $O(|\mathcal{A}|)$ row-softmax term is absorbed. Relative to residual-only PS, the extra GTA-PS overhead per DP plan step is therefore tightly bounded by $O(\log q_t + |\mathcal{S}|)$. For one Dyna simulated planning backup, the same expression holds with the Bellman-style term replaced by the TD-style host cost.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 5 GTA-PS Module

Precompute (once, $O(|\mathcal{S}|^3)$)
$P_0^{\mathrm{PS}} \leftarrow \sum_a \mathrm{diag}(\pi_0^{\mathrm{PS}}(\cdot, a))\, P^a$
$L_0^{out} \leftarrow I - P_0^{\mathrm{PS}}$; $L_0^{in} \leftarrow D_0^{in} - (P_0^{\mathrm{PS}})^\top$
$(L_0^{out} + \eta I)^{-1}$ via LU ▷ cached for all subsequent tri-solves
$(v_2, u_2, \mathrm{SLEM}) \leftarrow \textsc{Arnoldi}(P_0^{\mathrm{PS}}, (P_0^{\mathrm{PS}})^\top)$, $j = 15$
$\beta_0 \leftarrow \beta_1 \cdot (1 - \mathrm{SLEM} + \beta_2)$; choose $q_0 \in [K, |\mathcal{S}|]$
for each $s \in \mathcal{S}$: $\xi[s] \leftarrow |(\mathcal{T}V_0)(s) - V_0(s)|$
$\textsc{RefreshTau}()$ ▷ initial $\boldsymbol{\tau} = G\xi$; builds all queues

procedure $\textsc{GtaPolicyUpdate}(s, A^{\mathrm{PS}}(s, \cdot))$ ▷ $O(|\mathcal{A}|)$
  $\pi^{\mathrm{PS}}(s, \cdot) \leftarrow \mathrm{softmax}(b\, A^{\mathrm{PS}}(s, \cdot))$
end procedure

procedure $\textsc{TopologyUpdate}(s_{\mathrm{upd}}, \delta\pi^{\mathrm{PS}})$
  $w \leftarrow \sum_a \delta\pi^{\mathrm{PS}}(a)\, p(\cdot|s_{\mathrm{upd}}, a)$; $\delta P^{\mathrm{PS}} \leftarrow e_{s_{\mathrm{upd}}}\, w^\top$ ▷ rank-1, $O(\deg)$
  $u \leftarrow (L_t^{out} + \eta I)^{-1} w$; $v \leftarrow (L_t^{out} + \eta I)^{-\top} e_{s_{\mathrm{upd}}}$ ▷ tri-solves: $O(|\mathcal{S}|)$
  $\mathbf{f}^{out} \mathrel{+}= \frac{v^\top \mathbf{f}^{out}}{1 - v^\top w} u$ ▷ SM exact update, $O(|\mathcal{S}|)$
  $\delta\lambda_2 \leftarrow u_2(s_{\mathrm{upd}})(w^\top v_2)/(u_2^\top v_2)$; $\mathrm{SLEM} \mathrel{+}= \mathrm{Re}(\bar{\lambda}_2/|\lambda_2| \cdot \delta\lambda_2)$ ▷ $O(\deg)$
  if $t \bmod T_{\mathrm{arn}} = 0$ **then** ▷ $T_{\mathrm{arn}} = \lceil |\mathcal{S}|^{1/2} \rceil$
    $(v_2, u_2, \mathrm{SLEM}) \leftarrow \textsc{Arnoldi}(P_t^{\mathrm{PS}}, (P_t^{\mathrm{PS}})^\top)$
    $\beta_t \leftarrow \beta_1(1 - \mathrm{SLEM} + \beta_2)$
    $\mathbf{f}^{in} \leftarrow (L_t^{in} + \eta I)^{-1} \xi$ ▷ batched rebuild
    $\textsc{RefreshTau}()$
  end if
end procedure

procedure $\textsc{RefreshTau}$ ▷ $O(|\mathcal{S}|)$ two tri-solves; $O(|\mathcal{S}| + q_t)$ queue rebuild
  $\mathbf{f}^{\mathrm{out}} \leftarrow (L_t^{out} + \eta I)^{-1} |\xi|$; $\mathbf{f}^{\mathrm{in}} \leftarrow (L_t^{in} + \eta I)^{-1} \xi$
  for each $s \in \mathcal{S}$: $\tau[s] \leftarrow (1 - \alpha)|\mathbf{f}^{\mathrm{out}}[s]| + \alpha\, |\mathbf{f}^{\mathrm{in}}[s]|$
  Rebuild $\mathcal{Q}_\tau$ from $\{(s, \tau[s])\}_{s \in \mathcal{S}}$
  $\mathcal{F}_q \leftarrow \textsc{TopQ}(\mathcal{Q}_\tau, q_t)$; build $\mathcal{H}_q$ from $\{(s, \xi[s] + \beta_t \tau[s])\}_{s \in \mathcal{F}_q}$
  Rebuild $\mathcal{Q}_\xi$ from $\{(s, \xi[s] + \beta_t \tau[s])\}_{s : \xi[s] > 0}$
end procedure

procedure $\textsc{RewardUpdate}(\delta\mathbf{r})$ ▷ called after each reward shift
  $\mathbf{r} \mathrel{+}= \delta\mathbf{r}$; $\mathbf{f}^{out} \mathrel{+}= (L_t^{out} + \eta I)^{-1} \delta\mathbf{r}$; $\mathbf{f}^{in} \mathrel{+}= (L_t^{in} + \eta I)^{-1} \delta\mathbf{r}$
  for each $s \in \mathcal{S}$: $\xi[s] \leftarrow |(\mathcal{T}V_t)(s) - V_t(s)|$
  $\textsc{RefreshTau}()$
end procedure

procedure $\textsc{PopAndPrioritize}$ ▷ $O(\log |\mathcal{S}| + \log q_t)$
  Let s_ξ, s_τ be heads of $\mathcal{Q}_\xi$, $\mathcal{H}_q$ (if nonempty)
  return whichever has larger $\xi[s] + \beta_t \tau[s]$, removing it from all queues
end procedure

procedure $\textsc{PushPredecessors}(s, \Delta V)$ ▷ $O(|\mathrm{Pred}(s)| \log |\mathcal{S}|)$
  for $s' \in \mathrm{Pred}(s)$ **do**
    $\xi[s'] \leftarrow |(\mathcal{T}V)(s') - V(s')|$; $\mathcal{Q}_\xi.\textsc{Update}(s', \xi[s'] + \beta_t \tau[s'])$
  end for
  if $k \bmod T_\tau = 0$ **then** $\textsc{RefreshTau}()$
  end if ▷ $T_\tau = |\mathcal{S}|$
end procedure

The dense τ refresh, dense topology rebuild, Arnoldi sync, and reward-update terms are secondary amortized corrections to this dominant local plan-step term. If standalone τ refreshes are triggered every T_τ host backups, dense topology rebuilds every T_{topo} host backups, Arnoldi is rerun every T_{arn} host backups, and reward shifts arrive with characteristic scale T, then the full per-step DP cost

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

admits the upper bound

$$C_{\mathrm{DP}}^{\mathrm{full}}(s) \;=\; C_{\mathrm{DP}}(s) \;+\; O\Big(\frac{C_\tau}{T_\tau} \;+\; \frac{C_{\mathrm{topo}}}{T_{\mathrm{topo}}} \;+\; \frac{C_{\mathrm{arn}}}{T_{\mathrm{arn}}} \;+\; \frac{C_{\mathrm{shift}}}{T}\Big).$$

For a Dyna host agent, one real environment interaction is followed by K simulated planning backups. The base per-interaction cost, excluding periodic sync overhead, is

$$C_{\mathrm{Dyna}} \;=\; O\Big(|\mathcal{A}| \;+\; K \cdot \big[|\mathrm{Pred}(s)| \log |\mathcal{S}| + \log |\mathcal{S}| + \log q_t + |\mathcal{S}|\big]\Big),$$

where the $O(|\mathcal{A}|)$ term is the real-step TD update and model learning, and the K factor scales the local plan-step overhead. To this base cost, each interaction adds its share of periodic work, proportional to the $K+1$ host steps it contributes. Contributing the maintaining overhead makes the realized cost on each host step become:

$$C_{\mathrm{Dyna}}^{\mathrm{full}} \;=\; C_{\mathrm{Dyna}} \;+\; O\Big((K+1)\Big[\tfrac{C_\tau}{T_\tau} + \tfrac{C_{\mathrm{topo}}}{T_{\mathrm{topo}}} + \tfrac{C_{\mathrm{arn}}}{T_{\mathrm{arn}}}\Big] \;+\; \tfrac{C_{\mathrm{shift}}}{T}\Big).$$

The first three terms scale with K because both the real step and all K simulated backups advance the host-step counters for τ refresh, topology rebuild, and Arnoldi cadence. The reward-shift term is independent of K since shifts arrive relatively to the host step.

The checked-in implementation uses the heuristic estimate $T_{\mathrm{arn}} = \lceil \sqrt{|\mathcal{S}|} \rceil$ and $T_{\mathrm{topo}} = T_\tau = |\mathcal{S}|$. They should not be read as asymptotic complexity optima or grounded computational error handler.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Appendix C. Intuitions, Design Decisions, Limitations and Future Directions

C.1 Key decisions and their rationale

Why nonstationary reward is the chosen setting With stationary reward, standard PS runs to convergence. At convergence $\xi_t \to \mathbf{0}$, so $\mathbf{f}_t^{out} = (L_t^{out} + \eta I)^{-1}\xi_t \to \mathbf{0}$ and the topology signal vanishes identically (asymptotic-zero property). For deterministic shortest-path problems, Dijkstra's algorithm computes the optimal policy in $O(|\mathcal{S}| \log |\mathcal{S}| + |E|)$ – strictly superior. Though GTA-PS can excel in both settings, the contribution are not enough to compensate both the computational cost and the enormous potential of the new queue system.

Nonstationary reward with bottlenecks is the setting where GTA-PS dominates. When r_t shifts at frequency $1/T$, the planner has T steps before the next shift. A full PS sweep costs $O(|\mathcal{S}|^2|\mathcal{A}|)$; under budget $T \ll |\mathcal{S}|^2|\mathcal{A}|$, the Bellman wavefront has not reached all states and the reactive residual signal is zero at predecessor-blocked states. GTA-PS's topology signal is nonzero at all states immediately after a reward shift (Prop. 5.3 in the main text), so it can prioritize predecessor-blocked states before the wavefront arrives. The sub-$O(|\mathcal{S}|^2)$ multiplier ensures the economy, while the empirical efficiency outperform standard PS in numerous competing domains.

Why the resolvent, among the diffuser family? There are numerous choices for a diffuser operator, such as k-truncated smoothing $(I - \eta L)^k$, heat diffusion $\exp(-\tau L)$, the Laplacian pseudoinverse, and the regularized resolvent. GTA-PS uses the resolvent because it is both controllable and economical.

- Truncated smoothing has a hard finite horizon, heat diffusion is costly to update incrementally, and the pseudoinverse lacks explicit horizon control while depending on the Laplacian nullspace. In contrast, the resolvent $(L + \eta I)^{-1}$ has η naturally acts as a geometric discounting factor.
- Resolvent has a concise, exact localized update formula in B.2, which we believe would be nontrivial for heat diffusion or pseudoinverse to possess.

Why low-rank update assumption? A topology-policy change at state s_{upd} induces a row-local perturbation $\delta P_t = e_{s_{\text{upd}}} w^\top$. This is why low-rank resolvent identities are natural in the analysis: it adhere to learning process being local rather than global. However, this does not restrict the reward non-station capacity, as multiple localized shift can be batched in a vector shift facing host step.

Relative to competing topology-aware works. The same design choice also clarifies where GTA-PS differs from adjacent methods. Relative to standard PS and VaST [19], GTA-PS is proactive: the resolvent field is dense immediately after a reward shift, so distant states can enter the queue before the Bellman wavefront reaches them. Relative to TVI/FTVI [20], GTA-PS uses a soft, interruptible topology signal rather than an SCC batch schedule; it can be stopped after any backup and re-used after a reward shift without recomputing a decomposition. Relative to the successor representation [21], reward updates remain cheap while the topology operator is applied through sparse linear solves rather than through storage of a full dense successor matrix.

Two-queue architecture with top-q frontier. Because $\boldsymbol{\tau}_t = G\xi_t$ is dense (nonzero everywhere when $\xi_t \neq \mathbf{0}$), inserting it directly into the residual heap would crowd the queue with low-priority states, adding an excessive $O(|\mathcal{S}|$ cost multiplier. The simple solution is to limit the queue capacity by q. Additionally, the implementation uses two heaps: $\mathcal{Q}_\xi$ over the active residual set and $\mathcal{H}_q$ over a filtered top-q frontier extracted from the dense $\mathcal{Q}_\tau$. This *decouples the two signals*: the residual heap guarantees that states with large Bellman error remain eligible at all times, while the topology frontier injects proactive candidates without flooding the heap. Setting $q_t = |\mathcal{S}|$ recovers the unfiltered version analyzed theoretically.

Budget priority: residual-backups first. In POPANDPRIORITIZE, residual-reducing backups are mandatory for convergence, while topology-only backups (states with $\xi = 0$ but $\tau > 0$) are exploratory. Under tight budget, the implementation ensures that $\mathcal{H}_q$ never starves $\mathcal{Q}_\xi$. This prevents the topology signal from consuming budget on no-op Bellman backups at already-converged states.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

LU factorization threshold. For small state spaces, sparse LU factorization provides exact triangular solves at $O(|\mathcal{S}|)$ each and is the default path in the tabular experiments. In the current code the threshold is explicit: `_LU_THRESHOLD = 300`. Above that threshold the module switches to GMRES with a sparse-direct fallback. The exact crossover depends on sparsity structure, but the checked-in implementation address this cutoff cleanly.

C.2 Failed approaches

Column cache abandoned. An earlier design maintained an explicit $O(|\mathcal{S}|^2)$ column cache $\mathbf{g}_s = G^{out}_{\eta,t}(\cdot, s)$ (the s-column of the Green's operator) to accelerate incremental updates. This was abandoned in favour of the Sherman–Morrison dual tri-solve approach: the column cache introduces $O(|\mathcal{S}|^2)$ memory and requires maintenance of $O(|\mathcal{S}|)$ columns as the policy evolves, while the dual-solve approach achieves the same update expressiveness at $O(|\mathcal{S}|)$ per step without any persistent column cache.

Stale-column incremental strategy. An intermediate design proposed maintaining stale approximate columns with periodic refresh, balancing $O(|\mathcal{S}|)$ local cost against $O(|\mathcal{S}|^2)$ full recomputation. This was deemed unnecessary: the Sherman–Morrison formula provides exact updates at the same $O(|\mathcal{S}|)$ cost without accumulation of refresh lag.

Full recomputation of in-field per policy step. The in-field perturbation $\delta L^{in}_t = \mathrm{diag}(w) - w\, e^{\top}_{s_{\mathrm{upd}}}$ is not rank-1, so Sherman–Morrison does not apply. An early implementation attempted to track $\mathbf{f}^{in}$ incrementally through a generalised rank-$(d+1)$ update; the fill-in in the update formula made this costlier than periodic batch recomputation. The settled design rebuilds $\mathbf{f}^{in}$ in batch at Arnoldi sync points, accepting $O(|\mathcal{S}|)$ periodic cost amortised over T_{arn} steps.

Forced Arnoldi on every reward shift. An early version triggered a full SLEM recomputation (Arnoldi) at every reward shift. This was wasteful: the transition matrix P_t does not change at a reward shift (only $\mathbf{r}_t$ changes), so $\mathrm{SLEM}(P_t)$ and its eigendecomposition remain valid. The current design lets Arnoldi fire on its natural cadence T_{arn}, independent of reward shifts.

Forward-only ablation is not uniformly best. The repository keeps a dedicated forward-only comparison by setting $\alpha = 0$ inside the asymmetry sweeps (RQ4 and RQ6). Those scripts treat it as an ablation, not as the deployed default. This reflects the underlying design lesson: in directed environments, relying only on $\mathbf{f}^{\mathrm{out}}_t$ can miss upstream source–sink structure that becomes visible once the backward field $\mathbf{f}^{\mathrm{in}}_t$ is mixed in. The code therefore tunes a fixed α for the full GTA variants rather than collapsing the method to the forward-only special case.

LU fill-in at large $|\mathcal{S}|$. For very large state spaces, sparse LU factorization fill-in grows beyond memory or incurs prohibitive one-time cost. The GMRES fallback is present precisely for this regime. The checked-in benchmark scripts probe this overhead on FourRooms and on GARNET instances up to $|\mathcal{S}| = 200$; future work should push that scaling study further and characterize the LU-to-iterative crossover as a function of transition sparsity rather than only of $|\mathcal{S}|$.

C.3 Limitations

The current study is still limited to small-to-medium tabular domains. Although GTA-PS's mechanism is sound and grounded, its scaling frontier on much larger state spaces is not addressed. Our experiments also focus on localized positive reward shifts. Empirically, the method is not uniformly strong under every perturbation: highly adversarial, drift-heavy, or spark-like transports can distort the topology signal and weaken guidance quality. On the algorithmic side, we validate mainly DP and Dyna-style hosts, leaving broader orthogonal integrations such as Dyna-2 or uncertainty-aware planners. Finally, GTA-PS incurs real wall-clock overhead from topology refreshes and queue maintenance, so the present paper demonstrates superior replanning under fixed backup budgets rather than lower raw compute cost.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

C.4 Future work and potential improvements

Fast-shift asymptotics. In the regime $\rho = T/t_{\mathrm{mix}}(1/4) \ll 1$ (reward shifts arrive much faster than the chain can mix), it is conjectured that the mean per-episode advantage $\bar{A}$ grows as $1/\rho$. A formal scaling argument for this conjecture, together with empirical validation across the full K/t_{mix} ratio range in GARNET, would establish the precise operating regime where GTA-PS's benefit is maximised relative to its computational cost.

Integration with learned models. The current theoretical analysis (Sections 5, 7) is confined to the exact-model setting (known p). Extending operator-norm bounds, decision-accuracy guarantees, and the ergodic cumulative advantage theorem to the learned-model case ($\hat{p}_t$ with estimation error) would cover Dyna-style deployment, where model noise interacts with topology diffusion. The multi-step TD residual oracle (Appendix B.2) provides a practical interface; the formal gap remains.

SLEM as a continuous signal for other RL components. The SLEM serves here as a scheduling statistic for β_t. More broadly, SLEM and the resolvent-based topology potentials could inform exploration bonuses (intrinsic rewards proportional to $\tau_t(s)$ for under-visited states), model-learning budgets (allocating more environment interactions when SLEM detects bottleneck formation), or curriculum generation (reward-shift scheduling that targets the low-conductance regime where GTA-PS's advantage is largest).

GPU acceleration and batching. Dense vector updates, sparse matrix-vector products, and Arnoldi iterations admit batching and GPU execution.

Unified design for the topology policy. The Boltzmann policy with fixed temperature b is a pragmatic baseline that ensures ergodicity. Alternative topology-policy designs – such as learned attention policies that minimise SLEM, or topology policies designed to align with specific competitive baselines – could replace the softmax heuristic. The only invariant requirement is that the induced chain remains communicating with infinite-visit exploration.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Appendix D. Environments, Hyperparameters, and Experimental Protocol

D.1 Environment setup

As to surgically target each term, we dedicated our experiments to the two environments that bring full control over the MDP characteristics.

Minigrid Fourrooms [15] The environment is encapsulated in the tabular class `FourRoomsTabular` consists a grid navigation field, with four rooms connected restraints by middle walls, connected by narrow one-cell corridors and default action set size $|\mathcal{A}| = 4$. Under the default geometry the state space varies from 20 to 120 passable cells. The checked-in configs cover $k \in \{1, 4\}$ active-room settings for the majority of scripts and a separate $k = 2$ dedicated for fine-grain time-series observations. Optional asymmetry controls come from the `drift` and `one_way_prob` arguments – variations between them grant precise control over certain targeted statistics.

GARNET [4]. The synthetic benchmark is the tabular class `GarnetEnv`. The checked-in configs use $(|\mathcal{S}|, |\mathcal{A}|) = (75, 4)$ and $(200, 5)$, with branching factor 3, neighbor-rate `n_nbr_rate` controlling locality, and optional dedicated `reversible`, `bipartite`, and `reward_locality` switches. Several experiment scripts also instantiate ad hoc GARNET settings with $|\mathcal{S}| \in \{75, 200\}$. This is the main environment for controlled sweeps over mixing rate, reversibility, reward support, and solver overhead.

We construct various custom attributes on top of these base environments, such as mixing rate, reward span, reversibility factor, branching factor, and planning-budget effects under controlled nonstationarity.

D.2 Experiment Configurations

All reported curves are aggregated over independent random seeds. The main orchestrator script suggest running the experiment on at least 1000 seeds and ≥ 10 logical processors. Confidence bands are computed as 95% Student-t intervals over the seed means.

Hyperparameter tuning uses the standalone Optuna driver [2]. Our tuning trial uses 200 trials with `seed_offset`=100000 so that tuning seeds stay disjoint from the evaluation seeds, therefore avoid inducing selection bias.

For the deployed GTA-PS module, the numerical linear-solver tolerance is fixed across experiments: exact sparse LU is used for $|\mathcal{S}| \leq 300$, and otherwise the code switches to GMRES with `restart`$=$ 30, `maxiter`$=$ 500, and absolute tolerance `atol`$=10^{-4}$, falling back to `spsolve` if GMRES returns nonzero status.

For the Dyna host agent, an exploration rate $\epsilon = 0.2$ is maintained to escape stale local loops, where the agent sample the current greedy action with probability $1 - \epsilon$ and samples a random admissible action with probability ϵ.

D.3 Reward protocol

All environments use state-based rewards, but the reward pattern are designed to differ across setting purposes.

FourRooms. Rewards are not single-cell goals in the checked-in environment. Instead, `FourRoomsTabular` samples a Gaussian-mixture reward field: it chooses k active rooms, places `n_peaks_per_room` Gaussian bumps inside each selected room, samples bump heights from the room-specific `room_peak_ranges`, and normalizes the final vector to unit maximum. A reward shift simply resamples this field.

GARNET. The default reward generator is sparse over `reward_support` states with amplitudes drawn uniformly from $[0.5, 1.5]$. When `reward_locality=True`, those rewarded states are drawn as a BFS-local connected cluster from the union graph of the transition tensor rather than as independent global samples.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

D.4 Hyperparameters Tuning and Selection

The repository includes a standalone Optuna tuner [2]. The selected configuration minimizes the mean L_∞ value error to the exact DP solution over 30 tuning seeds, 4 reward shifts, $K = 20$ planning backups, and 50 host steps per shift. The exact hyperparameter search interval and tuned values are in the supplement code.

D.5 Baselines and fair-comparison controls

The checked-in experiment harness instantiates three method variants:

- `standard_ps`: the Moore-style prioritized-sweeping baseline [40], using Bellman residual magnitude as the queue key.
- `gta_base`: GTA-PS with fixed α and fixed β.
- `gta_beta`: GTA-PS with fixed α and adaptive β_t.

All three variants are implemented through the unified priority-based planning interface B.3. This lets us run them under the exact same host setting, MDP instance, backup budget, and evaluation protocol, with no method-specific entanglement. The differences can only happen in their inner states themselves.

D.6 Why no baselines apart from Standard-PS?

We use Standard-PS as the sole direct baseline because it is the only comparison that matches our assumption set and intervention scope without introducing additional confounds.

Similar works discussed in the paper, including TVI [20], VaST [19], Dyna-2 [54], or Generalized Prioritized Solver (GPS) [64] are therefore are not really click as direct baselines.

Under the communicating-MDP assumption, TVI effectively reduces to a Standard-PS update schedule. VaST introduces supervised representation learning, which is orthogonal to our fixed-MDP priority-design setting. Dyna-2 and concerns host-agent architecture and model-based decision organization, whereas our study focuses only on the management of sequential value backups inside a given planning interface.

Generalized prioritized solvers [64] is the most close candidate, with a priority score of Value-plus-error. However, the issue of stale value-function entries (by nonstationary rewards) misleading the queue is clear logically. In one hand, we have empirically observed that value-function can barely adapts in a reasonable pace E.2. In the other hand, GPS was shown to degrade under environments with a high dose of stochasticity, in which our method holds close performance from standard PS 6. We instead use these methods as major contextual guidances, which aids our grasp in the setting of nonstationary RL.

D.7 Computational details and optimizations

We implements some nontrivial optimizations to further bring the wall-clock performance of GTA-PS down:

Sparse representation. The topology module stores the policy-induced systems in SciPy sparse form. In `GtaPS`, P_π is converted to CSR, near-zero entries below 10^{-12} are dropped, and the forward system is assembled as $(1+\eta)I - P_\pi$. The in-field system uses `in_laplacian(P_pi)` with an additional diagonal stabilizer of $\eta + 10^{-6}$ in the initialization and lazy-rebuild path.

Solver selection in the deployed module. The experiment implementation uses the threshold `_LU_THRESHOLD = 300`. For $|\mathcal{S}| \leq 300$, both forward and backward systems are factored with `scipy.sparse.linalg.splu` and then reused for all subsequent solves. For $|\mathcal{S}| > 300$, the module calls `gmres` with `restart=30`, `maxiter=500`, and `atol=1e-8`, with a fallback to `spsolve` if GMRES reports nonzero status. The separate helper functions in `utils/spectral.py` also expose LU/GMRES resolvent routines and the ARPACK-based SLEM estimator used by the topology layer.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 2: Empirical wall-clock planning cost, reported as median milliseconds over 120 repetitions. GTA-PS incurs a moderate but stable runtime multiplier from topology refreshes and dual-queue maintenance.

Environment	$\lvert\mathcal{S}\rvert$	K	Standard-PS	GTA-base	GTA-β	Overhead
FourRooms	40	10	4.10	9.73	9.31	$2.37\times$ /$2.27\times$
GARNET	75	18	18.79	48.41	48.85	$2.58\times$ /$2.60\times$
GARNET	200	50	51.20	112.39	114.00	$2.20\times$ /$2.23\times$

Queue and predecessor maintenance. The module rebuilds the dense top-q frontier with `np.argpartition`, uses Python `heapq` heaps for both residual and topology queues, and filters predecessor edges at threshold 10^{-2} when building the cached predecessor sets. This threshold is used both when the oracle graph is first cached and when empirical model rows are updated online.

D.8 Additional experiment results.

We spend our remaining supporting sweeps to this subsection. Together, Figs. 4–7 provide the broader experimental ground behind the main text. They show how to calibrate topology strength, how tolerable backward mixing is, and in which stochastic or navigational regimes the topology signal is most effective.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

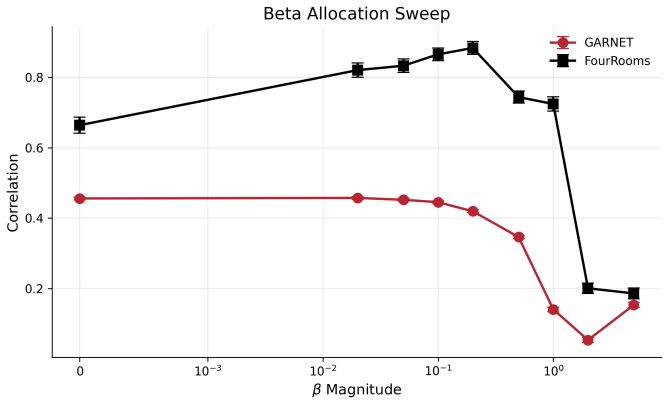


Figure 4: Figure 4 studies a single reward-shift event from a zero-initialized reward field in two environments: GARNET with $|\mathcal{S}| = 75$ and FourRooms with $|\mathcal{S}| = 40$ when varying over calibration weight β. The vertical axis is Spearman correlation ρ between the initial topology ranking and the realized pop order during the first K backups. Larger values indicate that the topology field is better aligned with the backups the planner actually performs. Moderate β gives the strongest agreement between the topology field and the realized backup order, peaking at $\beta = 0.02$ in GARNET and $\beta = 0.1$ in FourRooms. Larger topology gain sharply degrades the ordering quality, which complements the smallness condition in Prop. 5.1.

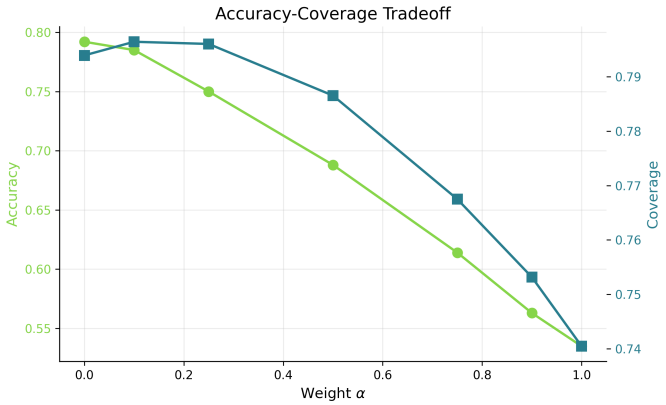


Figure 5: Figure 5 is a GARNET design that calibrates over fixed $\alpha = \alpha_t$. The left vertical axis reports top-1 accuracy, namely whether the state ranked first by the topology score matches the oracle state ranked first by the greedy propagation surrogate. The right vertical axis reports how much of the oracle's high-priority predecessor set is recovered by the top-ranked topology states. The results favor the use of small α, which favor out-degree Laplacian signal, prioritizing proactive backward propagation.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

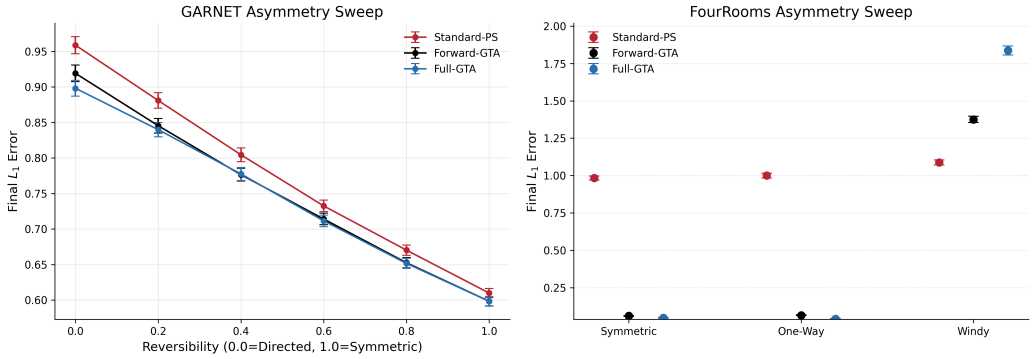


Figure 6: Figure 6 reports shift-end L_1 value error after repeated replanning under a fixed budget. In the left panel, the GARNET horizontal axis is the reversibility coefficient, where 0 is maximally directed and 1 is exact symmetry (reversible chain); lower values therefore correspond to more strongly directional random dynamics. In the right panel, FourRooms is perturbed by three transport modes: symmetric dynamics, a one-way corridor bias, and a windy drift field. GTA-PS remains competitive in strongly directed or one-way regimes, but the benefit is not monotone in asymmetry: drift-heavy transport can weaken or reverse the gain.

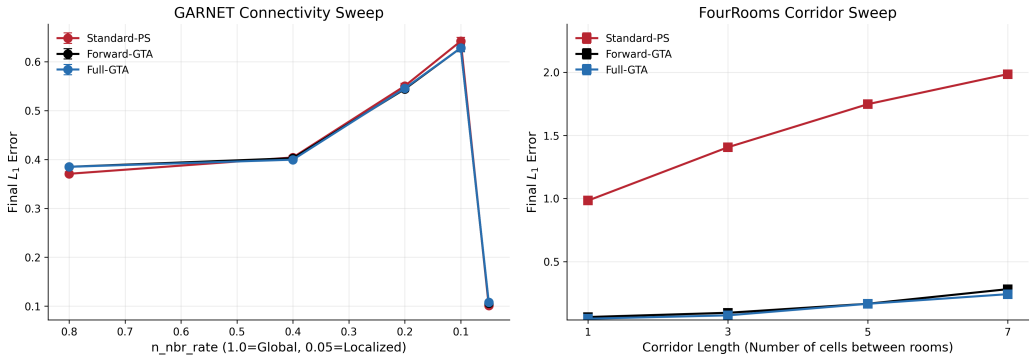


Figure 7: Figure 7 also measures shift-end L_1 value error under the same repeated-shift budgeted replanning protocol. In the left GARNET panel, the horizontal axis is the neighbor-rate parameter `n_nbr_rate`, which controls how globally connected the random transition graph is; moving right-to-left corresponds to increasingly localized connectivity.The right FourRooms panel ablates over the depth ò the bottleneck corridor, measured in grid cells. Lower error therefore means that the planner is coping better with either random sparsification in GARNET or geometric bottlenecking in FourRooms. In corridor-heavy FourRooms layouts, topology-aware prioritization remains substantially stable than Standard-PS, while excels in long-corridor bottenecks.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Appendix E. Miscellaneous Supplement

E.1 Dyna/DP + GTA-PS pseudocode

To deeply specifying the algorithms, we provide detailed pseudocodes describing how the host agent interacts with the module in each setting of DP and Dyna B.1

Algorithm 6 DP + GTA-PS

Require: $A_0^{\mathrm{PS}}(s,a) \leftarrow \gamma \sum_{s'} p(s'|s,a)\, V_0(s') - \max_{a'} \gamma \sum_{s'} p(s'|s,a')\, V_0(s')$; module temperature $b > 0$; $\pi_0^{\mathrm{PS}}(a|s) \leftarrow \mathrm{softmax}(b\, A_0^{\mathrm{PS}}(s,\cdot)),\ \forall s$

while not converged **do**
  if RewardShifted() **then**
    $r \mathrel{+}= \delta\mathbf{r}$; RewardUpdate($\delta\mathbf{r}$)
  end if
  for $k = 1$ **to** K **do** ▷ K Bellman backups available per replanning
    $s \leftarrow$ PopAndPrioritize()
    if $s = \varnothing$ **then break**
    end if
    $\Delta V(s) \leftarrow$ BellmanBackup(s, P, r, V) ▷ host backup, $O(|A| \cdot \deg)$
    UpdateValue($s, V(s)$) ▷ sync current host value into the module
    $A^{\mathrm{PS}}(s,a) \leftarrow \gamma \sum_{s'} p(s'|s,a)\, V(s') - \max_{a'} \gamma \sum_{s'} p(s'|s,a')\, V(s')$ for each a ▷ $O(|A| \cdot \deg)$
    NotifyBackup($s, A^{\mathrm{PS}}(s,\cdot), \Delta V(s)$) ▷ encapsulates local policy/topology update; no dense queue refresh
    PushPredecessors($s,\ \Delta V(s)$)
  end for
end while

Algorithm 7 Dyna (one-step TD) + GTA-PS

Require: module temperature $b > 0$; TD learning rate $\eta_{\mathrm{td}} \in (0,1]$; $A_0^{\mathrm{PS}}(s,a) \leftarrow Q_0(s,a) - \max_{a'} Q_0(s,a')$; $\pi_0^{\mathrm{PS}}(a|s) \leftarrow \mathrm{softmax}(b\, A_0^{\mathrm{PS}}(s,\cdot)),\ \forall s$

while not converged **do**
  $(s,a,r,s') \leftarrow$ RealStep() ▷ interact with environment; $a \sim \pi_t$ (host policy)
  Update $\hat{p}(\cdot|s,a), \hat{r}$ from (s,a,s') ▷ model learning; uses empirical model
  $\xi_s \leftarrow r + \gamma \max_{a'} Q(s',a') - Q(s,a)$; $Q(s,a) \mathrel{+}= \eta_{\mathrm{td}}\, \xi_s$
  UpdateModel($\hat{p}, \hat{r}$); UpdateValue($s, \max_a Q(s,a)$)
  PushPredecessors($s, |\xi_s|$) ▷ real step activates local residuals through the module interface
  if RewardShifted() **then** RewardUpdate($\delta\mathbf{r}$)
  end if
  for $k = 1$ **to** K **do** ▷ K simulated planning backups available
    $s \leftarrow$ PopAndPrioritize()
    if $s = \varnothing$ **then break**
    end if
    $a \leftarrow \arg\max_{a'} Q(s,a')$
    $(r_{\mathrm{sim}}, s'_{\mathrm{sim}}) \sim \hat{p}(\cdot|s,a)$
    $\xi_s \leftarrow r_{\mathrm{sim}} + \gamma \max_{a'} Q(s'_{\mathrm{sim}}, a') - Q(s,a)$; $Q(s,a) \mathrel{+}= \eta_{\mathrm{td}}\, \xi_s$
    UpdateValue($s, \max_a Q(s,a)$)
    $A^{\mathrm{PS}}(s,a) \leftarrow Q(s,a) - \max_{a'} Q(s,a')$ for each a
    NotifyBackup($s, A^{\mathrm{PS}}(s,\cdot), |\xi_s|$) ▷ encapsulates local policy/topology update; no dense queue refresh
    PushPredecessors($s, |\xi_s|$)
  end for
end while

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

E.2 Additional ablations

We report a focused subset of the ablation study, in which highlights some influential insights, as additional supplements for the appendix. These ablations isolate short-horizon error tracking, host-level reward accumulation, and the effect of the mixing-time threshold. The remaining ablations, implementation details, and extended results are provided in the code supplement.

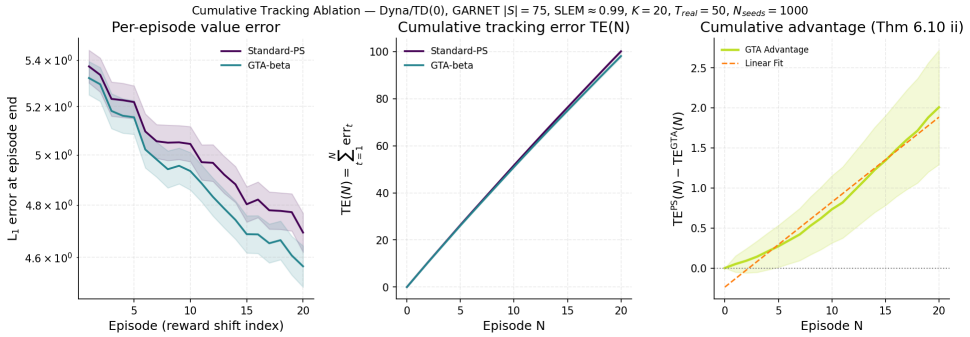


Figure 8: GARNET cumulative tracking ablation. The left panel shows per-episode value error, the middle panel accumulates the tracking error over episodes, and the right panel plots the theorem-facing cumulative advantage. GTA-beta reduces the episode-end error faster than Standard-PS, and the gap widens rather than collapsing. The cumulative tracking error $\mathrm{TE}(N)$ gap increases throughout the series, empirically backs A.3. The advantage curve remains close to linear with a positive slope, matching the intended interpretation of 5.1. This is the cleanest evidence that GTA-PS is doing useful work when reward shifts are frequent and the planner must repeatedly recover from fresh residual fields.

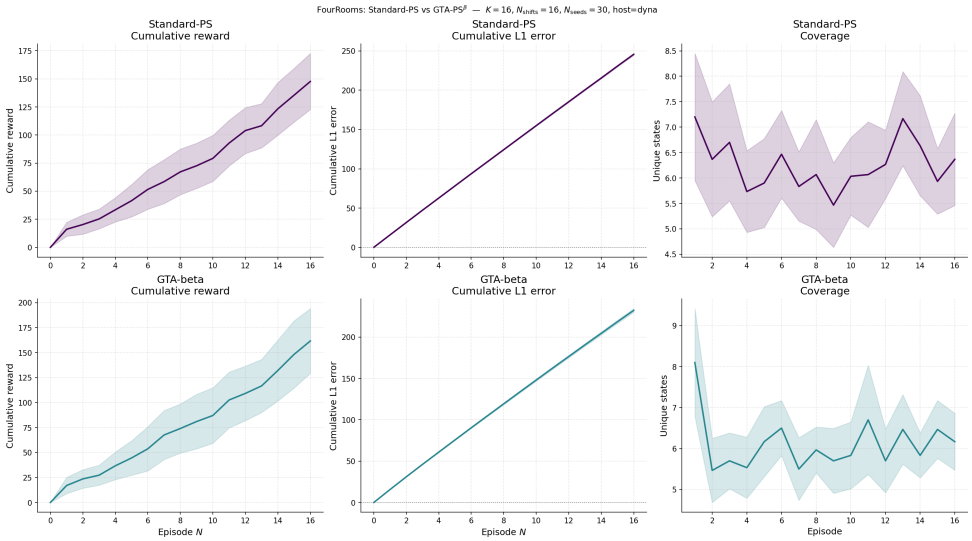


Figure 9: FourRooms ablation under a Dyna host. The top row reports Standard-PS and the bottom row reports GTA-PS$^{\beta}$ on the same seeds, budgets, and shift schedule. GTA-PS$^{\beta}$ achieves higher cumulative reward while simultaneously lowering cumulative ℓ_1 error, so the improvement was multi-faceted. The coverage traces remain in a comparable band for both methods, which shows that GTA-PS does not exploit aggressively. In the corridor-heavy FourRooms geometry, this is the right signal to look at because the topology module should help recover from bottlenecked propagation.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

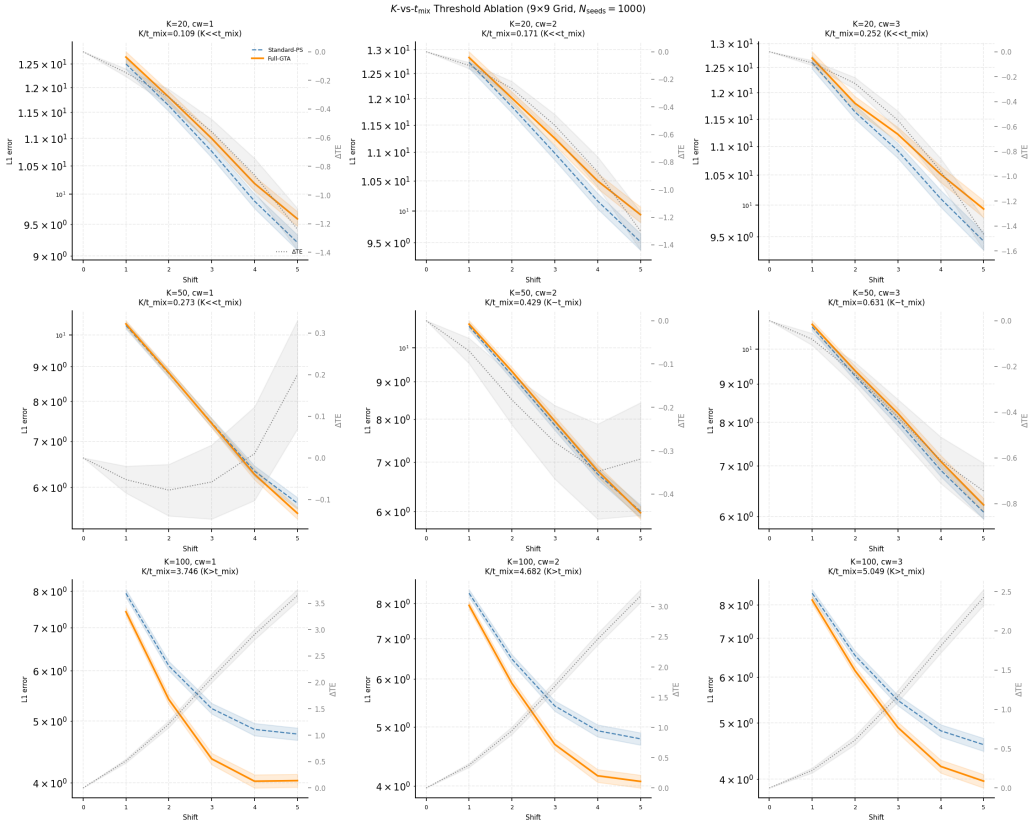


Figure 10: Threshold test against the K versus t_{mix} claim. Each panel fixes $K = 20, 50, 100$ and varies the corridor width, which changes the mixing scale and therefore the ratio K/t_{mix}. When $K \ll t_{\text{mix}}$, Standard-PS performs significantly worse than Standard-PS. As the ratio increases, the curves cross and the full GTA variant becomes competitive or superior. We can see that GTA-PS performs well over small corridors, empirically backs the intuition of proactiveness. GTA-PS also excels when given more compute, enabling scaling capabilities.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: Geometric potential operator as preconditioner and SLEM scheduler are the two fresh contributions of the paper. The experimental scope is tabular non-stationary domains, and the main result is that GTA-PS significantly reduces the number of *planning step* needed for single-shift convergence and recovery after a reward shift, compared to standard PS, agnostic to DP and Dyna settings.

   Guidelines:

   - The answer [NA] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [NA] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper discusses the limitations of GTA-PS, including the idealistic theoretical analysis to tabular, fully observed MDPs and the lack of more computation intensive experiments.

   Guidelines:

   - The answer [NA] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: The paper provides the full set of assumptions for the theoretical results, including the hard-set prior assumptions and individual assumptions for each result. The paper also provides complete and correct proofs for the theoretical results, with most of the details in the appendix.

   Guidelines:

   - The answer [NA] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper provides detailed descriptions of the experimental setup, including the tabular non-stationary domains, the algorithms compared, and the metrics used. This information is sufficient to reproduce the main experimental results.

   Guidelines:

   - The answer [NA] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We submit our experiment code via OpenReview and are open to share via direct communication.

   Guidelines:

   - The answer [NA] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Detailed descriptions of the experimental setup, including the environments,the algorithms compared, the hyperparameters, and the metrics used, are provided in the appendix.

   Guidelines:

   - The answer [NA] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [NA]

   Justification: The paper does not include experiments that require statistical significance reporting.

   Guidelines:

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

- The answer [NA] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [NA]

   Justification: The paper does not include experiments that require precise reporting of compute resources.

   Guidelines:

   - The answer [NA] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?

   Answer: [Yes]

   Justification: The research conducted in the paper adheres to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [NA] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No] , they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [NA]

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Justification: This is not the kind of work that has direct societal impacts, either positive or negative.

Guidelines:

- The answer [NA] means that there is no societal impact of the work performed.
- If the authors answer [NA] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: The paper does not include experiments that require safeguards for responsible release of data or models.

Guidelines:

- The answer [NA] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper properly credits the creators of all assets used and respects the licenses and terms of use.

Guidelines:

- The answer [NA] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [NA]

    Justification: The paper does not introduce new assets.

    Guidelines:

    - The answer [NA] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

    - The answer [NA] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: The paper does not involve crowdsourcing nor research with human subjects.

    Guidelines:

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

- The answer [NA] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [NA]

    Justification: The paper does not involve the use of LLMs as any important, original, or non-standard components.

    Guidelines:

    - The answer [NA] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

Confidential reviewer copy. This manuscript is submitted to the 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Unauthorized sharing, redistribution, or disclosure is strictly prohibited.